

ZPR PWr – Zintegrowany Program Rozwoju Politechniki Wrocławskiej

Web System Programming

W02 i W03 – HTML5 – elements and
attributes

Syllabus

- HTML5
 - HTML document and header
 - Elements **<meta>**
 - Comments
 - Text Formatting
 - Lists:
 - Unordered
 - Ordered
 - Definitions
 - Tables
 - Images on pages
 - Forms and their fields, field types, part of attributes
 - New form field types
 - Internal and external linking
 - New document structure elements
 - Elements: **<iframe>**, **<canvas>**, **<svg>**, **<video>**, **<audio>**, **<object>**, **<embed>**
 - Best Practices
 - Special characters

What is an HTML element?

- An HTML element is defined by: start tag, content, and end tag:

<tagName> contents **</tagName>**

- **An HTML element** is all content: from the start tag to the end tag:

<h1> Heading 1**</h1>**

<p> My first paragraph. **</p>**

- Important Notes:
 - Never skip the end tag
 - Some HTML elements will display correctly even if the end tag is missing. However, in such cases, unexpected results and errors may appear.
 - There are tags that do not have an end tag.
 - Some HTML elements have no content (for example, **<hr>**,**
**). These elements are called empty elements. Empty elements do not have an end tag.

Documents HTML

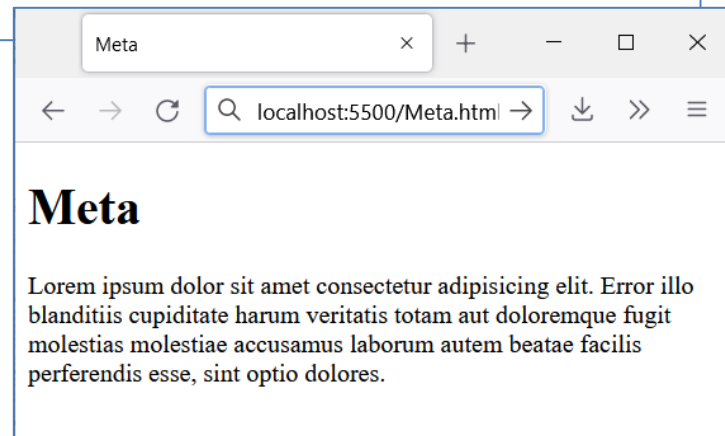
- All HTML documents can currently be used as document type: **<!DOCTYPE html>** which determines the document type and helps browsers that can view web pages.
- It only needs to be on the side once, at the very top (before the other ingredients). This entry is not case-sensitive.
- The HTML document itself begins with **<html>** and ends with **</html>**
- The document has the header (the content of it is not presented on the page) - it is between **<head>** and **</head>**
- The visible part of the HTML document is between the tags: **<body>** and **</body>**

Elements `<meta>`

- The `<meta>` elements are used by the browser to set some global properties
 - It should be used **in the header** and applies to the **entire** document.
- The content of `<meta>` elements is often used for cataloging web pages by search engines.
- It is an **empty element**
- The **name** attribute identifies the type of the meta element. Depending on its value, it changes the meaning of the **content** attribute. They are, in a way, a name-value pair.
- The meaning of the attribute **content** for **name** equal to (written in quotation marks):
 - `"keywords"` - keywords describing the page, separated by a comma
 - `"description"` - a multi-line description of the page in the form of sentences. This text is sometimes displayed as part of search results and is used in search engines.
- These elements are **not visible** on the website.

Elements <meta> - an example Meta.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="keywords" content="html5, tag meta, example">
  <meta name="description" content="The file contains an example of using a meta tag.
  inside the body the is only the lorem text. ">
  <title>Meta</title>
</head>
<body>
  <h1>Meta</h1>
  <p>Lorem ipsum dolor sit amet ...</p>
</body>
</html>
```



Horizontal line, text break

- HTML tags are not case-sensitive
 - `<html>`, `<HTML>`, `<Html>`, `<html>` - all are html tag
 - The HTML standard does not require lowercase tags, but the W3C recommends that you use **lowercase** (requires lowercase for more strict document types such as XHTML)
 - You can use the minus '-' as a word separator in the tag.
 - Eg. `<popup-info>`
 - There are no tags with minus in standard HTML5 tags
- By default, the browser converts the string of whitespace (space, `<enter>`, `<tab>`) into one whitespace and during rendering, depending on the possibility of fitting the word in the line, it converts this whitespace to one space or to a new line.
- The special character ` ` puts a "hard" space, which is not treated as white space, but as another character of a word:
 - There will be no line break at this point
 - You can visually make a larger space between words with a few codes ` ` .
- The empty `<hr>` tag means a **horizontal line** below the text.
 - It is deprecated, CSS should be used instead
- The empty `<br>` tag indicates a text break at this point (by default, the browser decides where to break a line within a paragraph).
 - Also not a very recommended way to force formatting
- Both tags **do not** define the content or structure and are therefore a sign of **ugly code**.

HTML comments

- You can add comments to the HTML source by using the following syntax:

```
- <!-- This is a comment -->
```

- Attention! Comments are not displayed by the browser, but can help document the HTML source code.

```
<!-- This is a comment-->
```

```
<p>This is a paragraph.</p>
```

```
<!-- This is a next comment -->
```

- Comments can be used to hide content. This can be helpful if you temporarily hide content:

```
<!-- This is a comment-->
```

```
<!-- <p>This is a paragraph.</p> -->
```

```
<!-- This is a next comment -->
```

- You can also hide more than one line, everything between the markers will be hidden from displaying. <!-- -->

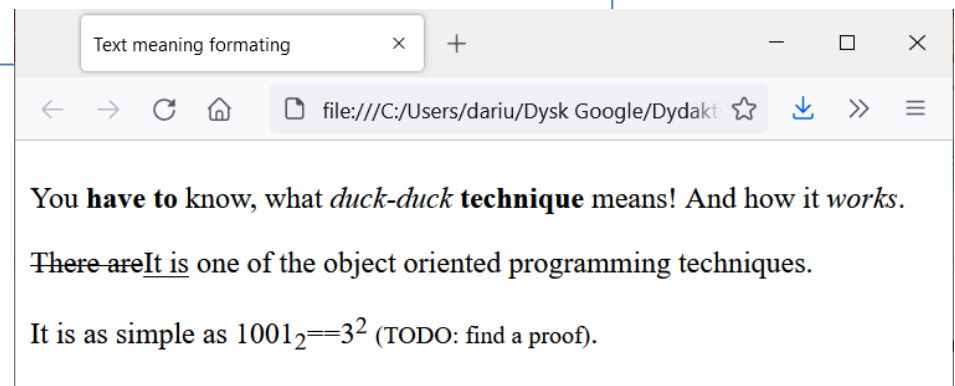
Text formatting tags

- Text formatting tags can be divided into two sets:
 - Forcing a specific formatting (not recommended unless, for example, some technical standard requires the use of bold or italic text):
 - **** - the text should be bold
 - **<i>** - the text should be italicized
 - Determining the meaning (e.g. emotional) of the text, semantic, the default behavior of which as shown below can be changed by CSS:
 - **** - important text, will make the text bold
 - **** - emphasized, underlined text will be displayed in italics
 - **<small>** - smaller font
 - **<mark>** - selected text
 - **** - deleted text (it will be crossed out)
 - **<ins>** - inserted text (when we want to show a change in relation to the previous version) will be underlined
 - **<sub>** - subscript
 - **<sup>** - superscript

Formatting tags - an example of use

- HTML5 start removed (space saving), so will be on the following pages (TextFormats.html).

```
<title>Text meaning formating</title>
</head>
<body>
  <h1></h1>
  <p> You <strong>have to</strong> know, what <em>duck-duck</em>
    <b>technique</b> means! And how it <i>works</i>.
  </p>
  <p>
    <del>There are</del><ins>It is</ins> one of
    the object oriented programming techniques.
  </p>
  <p>
    It is as simple as  $1001_2 = 3^2$ 
    <small>(TODO: find a proof)</small>.
  </p>
</body>
</html>
```



Lists

- Unordered list: ****
- Ordered list: ****
- Nested items of such lists: **** (list item)
 - For unordered elements, they will be preceded by a dot (this can be changed with the **style** = `"list-style-type: listStyle"` attribute)
 - For ordered elements, elements will be preceded by consecutive natural numbers starting from 1 (can be changed by the attribute **start** = `"number"`)
- They are considered as block elements, so they will always be on the new line.
- Lists can be nested. The bullet symbol or numbering format is then changed to emphasize the level of the list.
- The definition list: **<dl>** contains pairs:
 - Defined term element: **<dt>**
 - Definition data: **<dd>**, text usually with large indentation.

An ordered list

- An ordered list begins with the `<ol>` tag. Each element of the list begins with `<li>`.

- The list entries will be numbered by default:

`<ol>`

`<li> milk </li>`

`<li> cocoa </li>`

`<li> sugar </li>`

`</ol>`

- The `<ol>` tag **type** attribute defines the tag type of the list item:

– **type** = "1" list numbered with decimals

– **type** = "A" uppercase numbered list

– **type** = "a" lowercase numbered list

– **type** = "I" Roman numbered list – uppercase

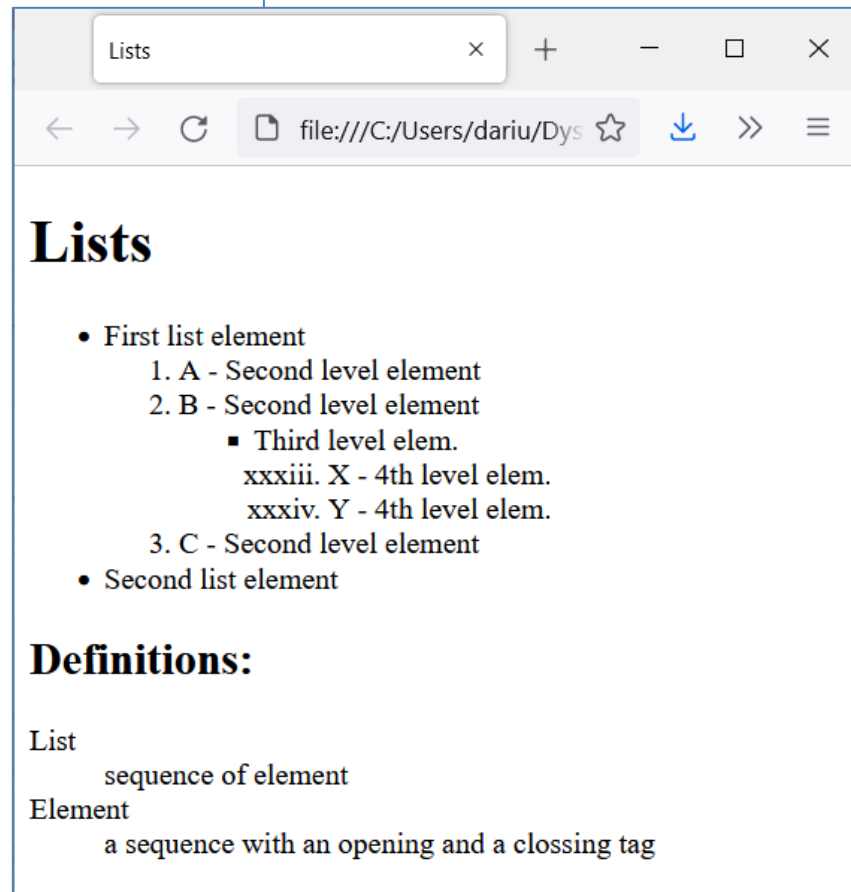
– **type** = "i" list numbered with roman numbers - lowercase

– Example: `<ol type = "A">`

An example of lists and nested lists - Lists.html

```
<title>Lists</title>
</head>
<body>
  <h1>Lists</h1>
  <ul>
    <li>First list element
      <ol>
        <li> A - Second level element</li>
        <li> B - Second level element
          <ul>
            <li>Third level elem.
              <ol start="33" type="i">
                <li>X - 4th level elem.</li>
                <li>Y - 4th level elem.</li>
              </ol>
            </li>
          </ul>
        </li>
        <li> C - Second level element</li>
      </ol>
    </li>
    <li>Second list element</li>
  </ul>
  <h2>Definitions:</h2>
  <dl>
    <dt>List</dt>
    <dd>sequence of element</dd>
    <dt>Element</dt>
    <dd>a sequence with an opening and a closing tag</dd>
  </dl>

</body>
</html>
```

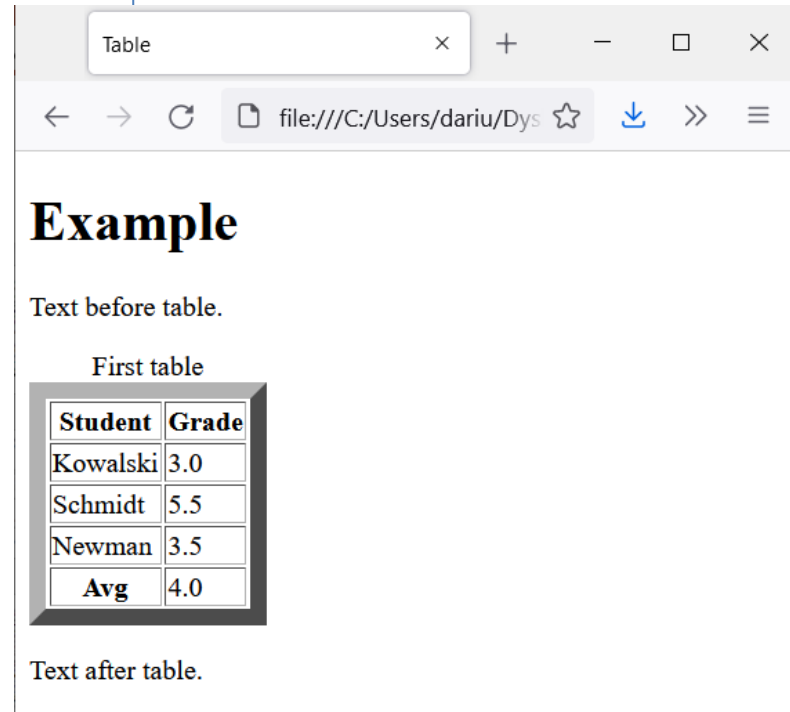


Tables

- **Tables** organize data into **rows** and **columns**. In the code, we arrange the table **row by row** and divide the rows into **cells**.
- The **<table>** element - defines an HTML5 table.
 - Attribute **border** = "X" specifies the width of the shading border around the entire table (obsolete)
 - There are many attributes to present the table (they will be used in the CSS topic)
- There can be three separate sections in the table (in the order):
 - **<thead>** element - the head, in which the column headings are defined.
 - **<tbody>** element - the body where the main table data will be
 - **<tfoot>** element - footer, e.g. for column summaries
- The **<tr>** element - a table's row, elements inside any of the section
- The **<th>** element - header cell, it should rather be inside the **<thead>** or **<tfoot>** section, or first element in a row
 - Typically bold font, centered text
- The **<td>** element - data cell, rather in the **<tbody>** section
 - Standard font, text moved to the left
- The **<caption>** element (**before the rest of the sections**, usually right after the opening **<table>** tag) specifies the table title.
 - Typically placed above the table.

Table – example - Table.html

```
<title>Table</title>
</head>
<body>
  <h1>Example</h1>
  <p>Text before table.</p>
  <table border="10">
    <caption>First table</caption>
    <thead>
      <tr>
        <th>Student</th>
        <th>Grade</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Kowalski</td>
        <td>3.0</td>
      </tr>
      <tr>
        <td>Schmidt</td>
        <td>5.5</td>
      </tr>
      <tr>
        <td>Newman</td>
        <td>3.5</td>
      </tr>
    </tbody>
    <tfoot>
      <tr>
        <th>Avg</th>
        <td>4.0</td>
      </tr>
    </tfoot>
  </table>
  <p>Text after table.</p>
</body>
</html>
```



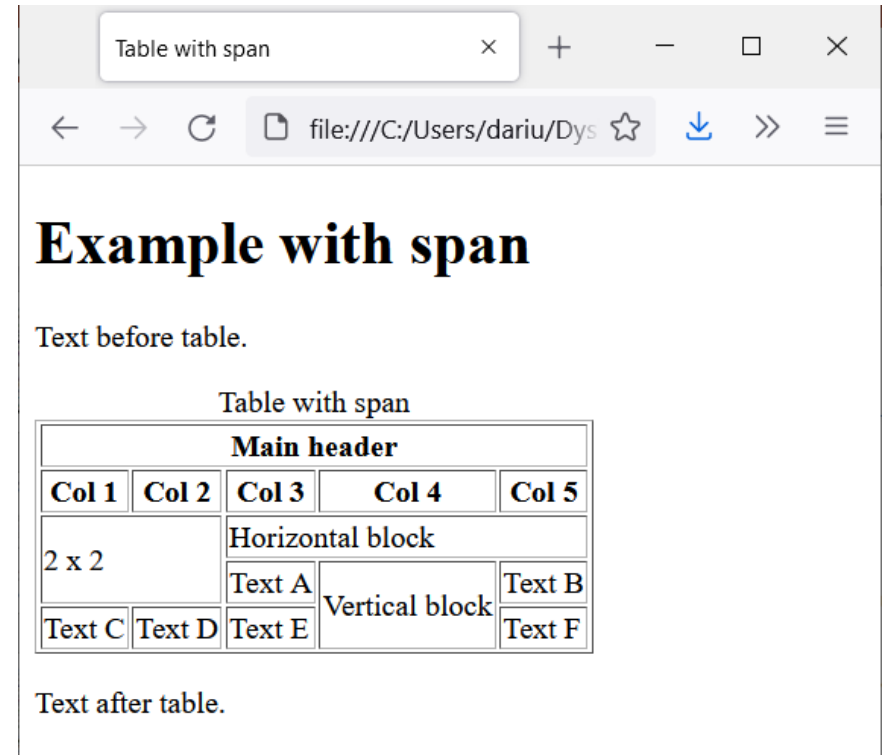
Tables - combine cells, rows and columns

- You can merge cells with data (or header data) by using the **rowspan** = "*X*" and **colspan** = "*Y*" attributes in **<th>** or **<td>** elements.
- The values of these attributes determine the **number of** rows or columns to be occupied by the cell.
- Subsequent **<td>** or **<th>** in the row refer to **next** cells.
- if **colspan** is used, the following lines use cell elements to identify the content of the **remaining, non-merged** cells.

- The standard rendering of a table measures the dimensions of the rows and columns to fit the displayed data.
- Through attributes, you can set the absolute dimensions (eg "*100px*") or relative sizes ("*30%*" of the page width) of the cell (that is indirectly of rows and columns)
 - Ultimately, all dividing lines must form a mesh (as in Excel, for example).

Combine cells - an example - TableWithSpan.html

```
<title>Table with span</title>
</head>
<body>
  <h1>Example with span</h1>
  <p>Text before table.</p>
  <table border="1">
    <caption>Table with span</caption>
    <thead>
      <tr>
        <th colspan="5">Main header</th>
      </tr>
      <tr>
        <th>Col 1</th><th>Col 2</th><th>Col 3</th>
        <th>Col 4</th><th>Col 5</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td colspan="2" rowspan="2">2 x 2 </td>
        <td colspan="3">Horizontal block</td>
      </tr>
      <tr>
        <td>Text A</td>
        <td rowspan="2">Vertical block</td>
        <td>Text B</td>
      </tr>
      <tr>
        <td>Text C</td><td>Text D</td>
        <td>Text E</td><td>Text F</td>
      </tr>
    </tbody>
  </table>
  <p>Text after table.</p>
</body>
</html>
```



Tables - other general information

- The contents of cells **can be anything**: text, link, image, other table, etc.
- Standard rendering will choose widths and heights so that everything fits.
 - With CSS You can enforce a certain minimum size by attributes for dimensions, inner and outer margins, frame thickness, etc.
- A table can be used to break a page, especially if you have disabled drawing of breaking edges.
 - It is **not advisable**, however, because such a page is often difficult to make *responsive*, i.e. responsive to changes in dimensions (rotation of the mobile screen, etc., reduction/enlargement of the window)

Images on the pages

- The tag **** is used to embed an image in a web page.
- Images are not technically inserted into the web page; images are linked to web pages.
- Image size - you can specify the width and height in one of two ways:
 - Using: **style**
``
 - Alternatively, you can use attributes: **width** and **height**
``
- The **width** and **height** attributes always define the width and height of an image **in pixels**.
- **Remark!** Always specify the width and height of the image. If you do not specify the width and height, the web page may flicker when the image is loaded.
- Which solution to choose: **width** and **height** or **style**?
 - Both are valid, but it is **suggested** to choose an attribute: **styles**.

Images on pages

- The tag `<img>` creates a place to hold the referenced image.
- The tag `<img>` is **empty**, has no closing tag, and contains only two required attributes:
 - **src** — specifies the path (URL) to the image,
 - **alt** — specifies alt text for the image that is used if the user for some reason cannot view it (due to a slow connection, an error in the `src` attribute, or if the user uses a screen reader).
- The attribute value should describe the image:
``
- If the browser cannot find the image, it will display the attribute value **alt**:
``
- **Remark!** A screen reader is a program that reads HTML code and allows the user to "listen" to content. Screen readers are useful for people who are visually impaired or have a learning disability.

Images on pages

- If the images are in a subfolder, specify the subfolder name in the attribute `src`:

```

```

- HTML allows animated GIFs: To use an image as a link, place a tag `<img>` inside the tag `<a></a>`:

```
<a href="default.html">  
  </a>
```

- You can use the CSS `float` property to place the image to the right or left of the text:

- ```
<p>
```

The image will be flown around by the text on the right.

```
<p>
```

The image will be flown around by the text on the left.

- **Notes on External Images:** External images may be subject to copyright. if you do not obtain permission to use it, you may be infringing copyright. In addition, external images cannot be controlled; someone can suddenly delete or change it.

## Images on pages – element `<picture>`

There are two main purposes of the element `<picture>`:

1. **Bandwidth** - If you have a small screen or device, it is not necessary to load a large image file. The browser uses the first element `<source>` with matching attribute values and ignores any of the following.
2. **Format support** – Some browsers or devices may not support all image formats. With this element `<picture>`, you can add images of all formats, and the browser will use the first format that it recognizes and ignore any of the following:

```
<picture>
 <source srcset="img_avatar.png">
 <source srcset="img_dog.jpg">

</picture>
```

**Remark!** The browser will use the first element `<source>` with matching attribute values and ignore any subsequent elements `<source>`.

# Website icon

- **Favicon** is a small image displayed next to the page title in the browser tab. You can use any image you like as your favicon. You can also create your own favicon on sites like <https://favicon.cc>
- **Remakr!** Favicon is a small image, so it should be a simple high-contrast image.

```
<!DOCTYPE html>
<html>
<head>
 <title>My first page</title>
 <link rel="icon" type="image/x-icon" href="/images/favicon.ico">
</head>
<body>
```

## Attribute **id** - identifier

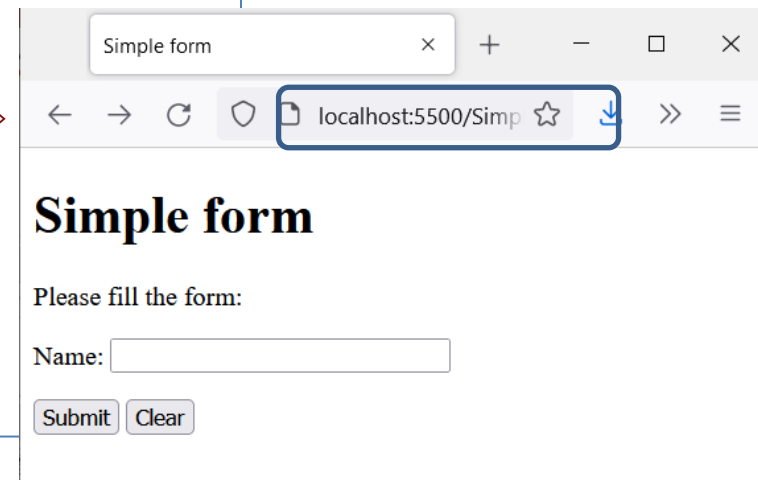
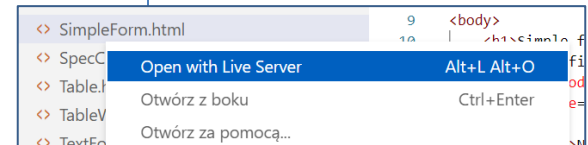
- On practice every element within HTML can have its own **unique identifier**.
  - Thanks to this, it is easy to reach for such an element in JS, CSS3, etc.
  - It can even be required in JS functions, various frameworks.
- It is assigned to an element using the **id** attribute, **id**="*idValue*".
- The ID is **case sensitive**.
- In principle, only whitespace is prohibited.
  - To be HTML4 compliant, it is not advisable to start the identifier with a number.
- You can use minus '-' or an underscore '\_' as a word divider.
  - A standard minus sign makes it difficult to automatically map the identifier to properties in programming languages (e.g. JavaScript).



# Forms and their fields

- Forms are used to collect information from users of a web page.
- Below is the code of a simple form with one field and its appearance.
- To observe the operation of the form in VS Code, it is worth installing the Live Server extension, open the page using this server and enter the address in the browser:  
<http://localhost:5500/SimpleForm.html>
- An Example: SimpleForm.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta http-equiv="X-UA-Compatible" content="IE=edge">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Simple form</title>
</head>
<body>
 <h1>Simple form</h1>
 <p>Please fill the form:</p>
 <!-- <form method="post" action="superweb.mydomain.com/userinfo.html"> -->
 <form method="post" action="http://localhost:5500/SimpleForm.html">
 <input type="hidden" name="securityCode" value="DE79W23">
 <p>
 <label for="idName">Name:</label>
 <input id="idName" name="Name" type="text" size="30" maxlength="50">
 </p>
 <p>
 <input type="submit" value="Submit">
 <input type="reset" value="Clear">
 </p>
 </form>
</body>
</html>
```



## Form - `<form>`

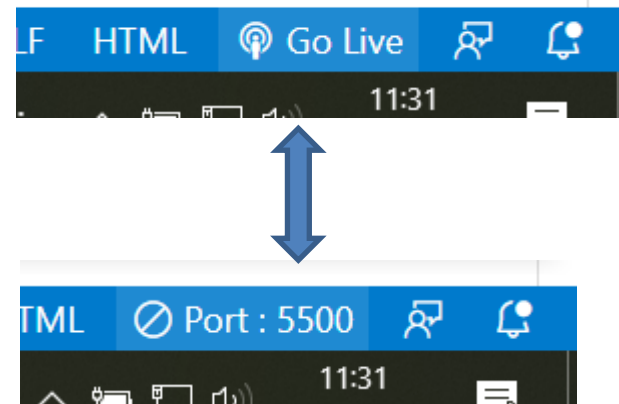
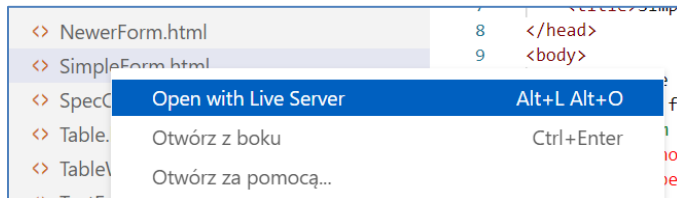
- The form is inside the element `<form>`
- The **method** attribute specifies how form data is sent to the web server.
  - The `"post"` value passes the form data in query string format in the body of the browser's POST request.
  - The value `"get"` passes the form data in query string format in the URL of the GET request sent by the browser.
- Attribute **action** of element `<form>` specifies the recipient of the **form handler** :
  - This can be a URL
  - This can be a JavaScript method
- Forms can contain visible and invisible components.
  - Visible components are e.a. clickable buttons and other graphical components that users use.
  - Invisible components, called **hidden fields**, store any text data, e.g. special security codes, data from previous pages of the form, etc.
- There can be **multiple forms on a page**. They are **independent** of each other and data from **only one** is converted into one HTTP request.

# Basic types of form fields

- Elements `<input>` specify the data to be provided to the recipient of the form.
- Element type `<input>` is specified by the `type` attribute.
- The `"hidden"` type contains hidden elements that must also have the `name` and `value` attributes. A pair of these values is sent in a query string.
- Type `"text"` of element `<input>` inserts a text box into the form so that the user can enter the data.
  - The `size` attribute specifies the **number of characters** that appears in the text box.
  - The optional `maxlength` attribute is the **maximum number of characters** you can type in a text box.
- Element `<label>`, i.e. a label, provides users with information about the purpose of a given element `<input>`.
  - It can have the attribute `for="idOfInput"`, indicating that it is a label for the element `<input>` with the identifier `idOfInput`.
- Type `"submit"` of element `<input>` inserts a button with the text of attribute `value` into the form.
- When you press the `"submit"` button, the form data is sent to the location specified in the `action` form attribute
- Type `"reset"` of element `<input>` inserts a button with the inscription with `value` attributes into the form, the pressing of which clears all elements of the form and restores them to their default values.

# Communication observation in VS Code – Live Server extension

- To observe HTTP communication, e.g. in a browser, a server must be running
  - It is not enough to open a file from a file explorer, because it is not an HTTP server.
- For example, in Visual Studio Code, the easiest way to install an extension called Live Server (extension ID: `ritwickdey.liveserver`)
- This server is not prepared to receive data from a POST query, but will simulate an attempt to handle such a request by answering the code 405 Method not Allowed.
  - But this will allow you to observe the headers and the body **of the request**.
- Startup can be done by opening the `.html` file accordingly
  - The default port is 5500.
- You can also start and stop the server in the lower right corner of VS Code. The URL is then, for example, `http://localhost:5500/SimpleForm.html`



# POST request – observation in the browser - SimpleForm.html

The diagram illustrates the process of submitting a form and observing the resulting POST request in the browser's developer tools. It consists of three browser window screenshots connected by arrows.

**Top Left Screenshot:** A browser window titled "Simple form" at `localhost:5500/Simp`. The form contains the text "Please fill the form:", a text input field with the value "Konieczny", and "Submit" and "Clear" buttons.

**Top Right Screenshot:** The same browser window after the form is submitted. The "Submit" button is highlighted with a red box. A blue arrow labeled "F12" points from this window to the bottom left window.

**Bottom Left Screenshot:** The browser's developer tools are open, showing the "Sieć" (Network) tab. A table of requests is displayed, with the first request highlighted in red:

Stan	Metoda	Domena	Plik
405	POST	localhost:5500	SimpleForm.html
404	GET	localhost:5500	favicon.ico

The status "405" and the method "POST" are highlighted in red. Below the table, the status bar shows "2 ządania", "Przesłano: 150 B / 926 B", "48 ms", and "DOMContentLoaded: 23 ms".

**Text:** For the POST request the response body is empty

# POST request and response

Sta	Met	Domena	Plik	Inicja...	Ty	Przes...	Rozmiar	Nagłówki	Ciasteczka	Żądanie	Odpowiedź
404	P...	local...	SimpleFor	docu...	htn	926 B	0 B	Filtruj nagłówki			
404	G...	local...	favicon.ico	Favic...	htn	w pa...	150 B				
								▶ POST http://localhost:5500/SimpleForm.html			
								Stan 405 Method Not Allowed ?			
								Wersja HTTP/1.1			
								Przesłano 926 B (o rozmiarze 0 B)			
								Zasada polecającego strict-origin-when-cross-origin			
								▼ Nagłówki odpowiedzi (271 B)			
								Access-Control-Allow-Credentials: true			
								Access-Control-Allow-Origin: http://localhost:5500			

Only when we know backend technology we will be able to handle such requests.

Currently the server returns an error 405 Method not Allowed

Wszystkie

HTML

CSS

JS

XHR

Czcionki

Obrazy

Media

WebSocket

Inne

Sta

Me

Domena

Plik

Inicja...

Ty

Przes...

Rozmiar

Nagłówki

Ciasteczka

Żądanie

Odpowiedź

Pomia

404

P...

local...

SimpleFor

docu...

htn

926 B

0 B

Filtruj parametry żądania

404

G...

local...

favicon.ic

Favic...

htn

w pa...

150 B

Zawartość żądania

Nieprzetworzone

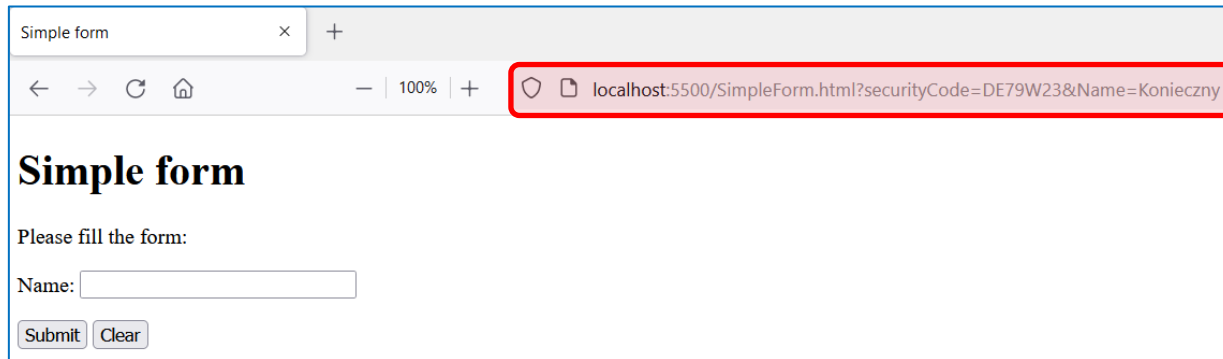
1

securityCode=DE79W23&Name=Konieczny

In the POST request **itself**, in the body, there are form fields in **query string** format.

# POST request and blank response body

- **Question:** Why don't we normally see a blank page when we submit a form on web pages using a POST request?
- **Answer:** The body of the answer is empty, but with proper handling we get a response with a different code, and in addition, in the header part there is a header "Location:", which the browser interprets as the need to execute a GET request to the specified URL.
- At the given address there is already a "normal" page containing HTML code.
- This means that form service is usually a minimum of two requests made by the browser
- Example sends the form data back to the same page (which may be correct because it sends a **different type** of request), but can send to a completely different address.
- If we wanted to handle this request, we would have to write support on the server (on a simple Live Server this cannot be easily done).
- You can also send data from a form to a method written in JavaScript, but then the **action** attribute has a different format (it is not a URL).
- Changing the method to **method="get"** will send the data **in the URL**:



The screenshot shows a web browser window with a single tab titled "Simple form". The address bar displays the URL `localhost:5500/SimpleForm.html?securityCode=DE79W23&Name=Konieczny`, which is highlighted with a red rectangular box. The page content includes the heading "Simple form", the text "Please fill the form:", a text input field labeled "Name:", and two buttons labeled "Submit" and "Clear".

## Form field `<textarea>` and field types of `<input>`

- Element `<textarea>` inserts a multi-line text field into the form.
  - The number of rows is determined by the `rows` attribute and the number of columns (i.e. characters per line) by the `cols` attribute.
- Type `"password"` of element `<input>` is used to enter a password in the form.
  - The data entered (password, credit card number, etc.) is "masked" most often by asterisk characters.
  - The request sends the correct field value, not asterisks or other masking characters.
- Type `"checkbox"` of element `<input>` allows users to make a choice.
  - When the checkbox is selected, a check mark appears inside the checkbox. Otherwise the checkbox is empty.
  - Checkboxes can be used individually and in groups. Checkboxes that are part of the same group have the same name (`name` attribute). Then their place in the form basically does not matter, although they should be visually close to each other.
  - Most often, the form has multiple checkboxes with the same name. Therefore, you should make sure that they all have different values; otherwise, the script on the server will not be able to distinguish them.
  - If you select several options in a query string, there will be many pairs where the name will be the same but different values



Form – field types `<input>` and field `<select>` with `<option>`

- Type `"radio"` of element `<input>` works similarly to the `"checkbox"` type, however, **only one** radio button from a group can be **selected** at the same time.
  - All radio buttons in the group have the same name (`name` attribute) but a different value (`value` attribute).
- Form element `<select>` creates drop-down lists.
  - The `name` attribute identifies the drop-down list.
- Element `<option>` adds an item to the drop-down list (it must be inside such an item).
  - Through attributes, you can set the possibility of multiple or only one value from the drop-down list.
  - By default, the selected value (text) becomes the value for the specified `name`.
  - This can be changed by adding the `value` attribute to the element `<option>`.
- Selection elements can have a `checked` attribute (without a sign equals `'='` and values) that indicates that the option is pre-selected.
- An attribute **without a value** is a notational **shortcut**. Indicates an attribute whose value is equal to the attribute name.
  - So the `checked` attribute means `checked="checked"`.
- If there are spaces or other special characters as the value, they will be specially encoded in the query string.

# Form – example ComplexForm.html (1/2)

```
<title>Complex form</title>
</head>
<body>
 <h1>Complex form</h1>
 <p>Please fill the form:</p>
 <!-- <form method="post" action="superweb.mydomain.com/userinfo.html"> -->
 <form method="post" action="http://localhost:5500/SimpleForm.html">
 <p>
 <textarea name="descr" cols="30" rows="3">Short description</textarea>
 </p>
 <p>
 <label for="idPass">Password:</label>
 <input id="idPass" name="Name" type="password" size="30" maxlength="50">
 </p>
 <p>
 <label>What elements you know?:</label>
 <label><input name="knowledge" type="checkbox" value="html5" checked>HTML 5</label>
 <label><input name="knowledge" type="checkbox" value="css3">CSS 3</label>
 <label><input name="knowledge" type="checkbox" value="js">JavaScript</label>
 </p>
</form>
```

## Complex form

Please fill the form:

Password:

What elements you know?: ☒ HTML 5 ☐ CSS 3 ☐ JavaScript

How would you rate your OOP knowledge?: ☐ 5 ☒ 4 ☐ 3

How would you rate this course?



## Complex form

Please fill the form:

Password:

What elements you know?: ☐ HTML 5 ☒ CSS 3 ☐ JavaScript

How would you rate your OOP knowledge?: ☒ 5 ☐ 4 ☐ 3

How would you rate this course?

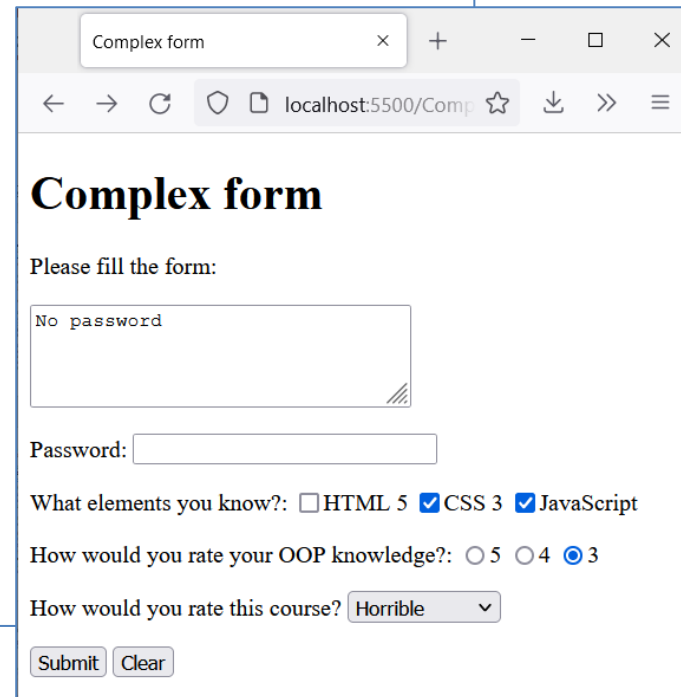


Zawartość żądania

```
1 descr=Probe+with+password+%22abc%22&Name=abc&knowledge=css3&oop=grade5&course=Bad+%26+worse
```

## Form – example ComplexForm.html (2/2)

```
<p>
 <label>How would you rate your OOP knowledge?:</label>
 <label><input name="oop" type="radio" value="grade5">5</label>
 <label><input name="oop" type="radio" value="grade4" checked>4</label>
 <label><input name="oop" type="radio" value="grade3">3</label>
</p>
<p>
 <label>How would you rate this course?
 <select name="course">
 <option>Excellent</option>
 <option>Good</option>
 <option value="canbe">Can be</option>
 <option>Bad & worse</option>
 <option value="0">Horrible</option>
 </select>
 </label>
</p>
<p>
 <input type="submit" value="Submit">
 <input type="reset" value="Clear">
</p>
</form>
</body>
</html>
```



Zawartość żądania

1	descr=No+password&Name=&knowledge=css3&knowledge=js&oop=grade3&course=0
---	-------------------------------------------------------------------------

New field types `<input>` for color and time

- All other types can have different graphical representation and functionality in different browsers.
  - Type `"color"` – to set the color
  - Type `"date"` – to set the date
  - Type `"datetime-local"` – set the date and time
    - There is an unrecommended `"datetime"` type that browsers currently ignore
  - Type `"month"` – setting only the month and year
  - Type `"time"` – setting the time
  - Type `"week"` – setting the number of the week in the year
- The default actions will be presented next, but through various attributes they can be changed:
  - e.g. whether to specify seconds in time, date range, etc.

# Example of use - NewTypesForm.html

```
<title>New types form</title>
</head>
<body>
 <h1>New types form</h1>
 <p>Please fill the form:</p>
 <form method="post" action="http://localhost:5500/NewerForm.html">
 <p>
 <label>Color:</label>
 <input id="idColor" name="Color" type="color" value="#FF00FF">
 </p>
 <p>
 <label>Date:</label>
 <input id="idDate" name="Date" type="date" value="2021-09-12">
 </p>
 <p>
 <label>Datetime-local:</label>
 <input id="idDateTimeLocal" name="DateTimeLocal" type="datetime-local">
 </p>
 <p>
 <label>Month:</label>
 <input id="idMonth" name="Month" type="month">
 </p>
 <p>
 <label>Time:</label>
 <input id="idTime" name="Time" type="time">
 </p>
 <p>
 <label>Week:</label>
 <input id="idWeek" name="Week" type="week">
 </p>
 <input type="submit" value="Submit">
 <input type="reset" value="Clear">
 </form>
</body>
</html>
```

New types form

Please fill the form:

Color:

Date:

Datetime-local:

Month:

Time:

Week:

Firefox 93.0

New types form

Please fill the form:

Color:

Date:

Datetime-local:

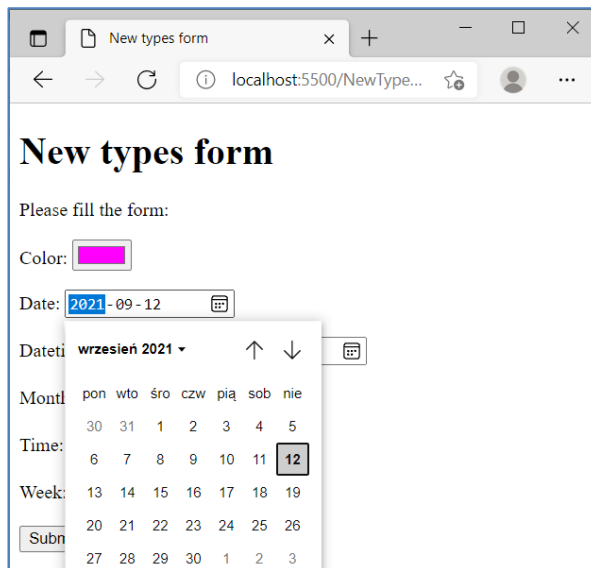
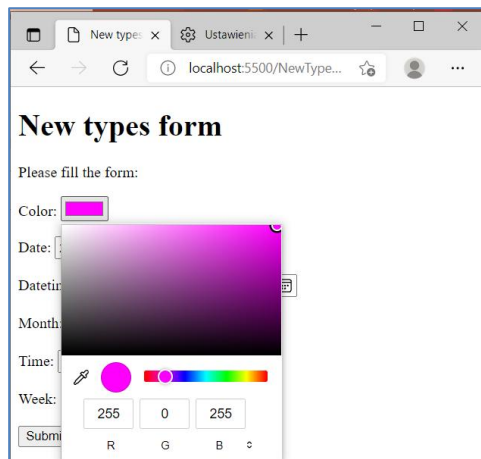
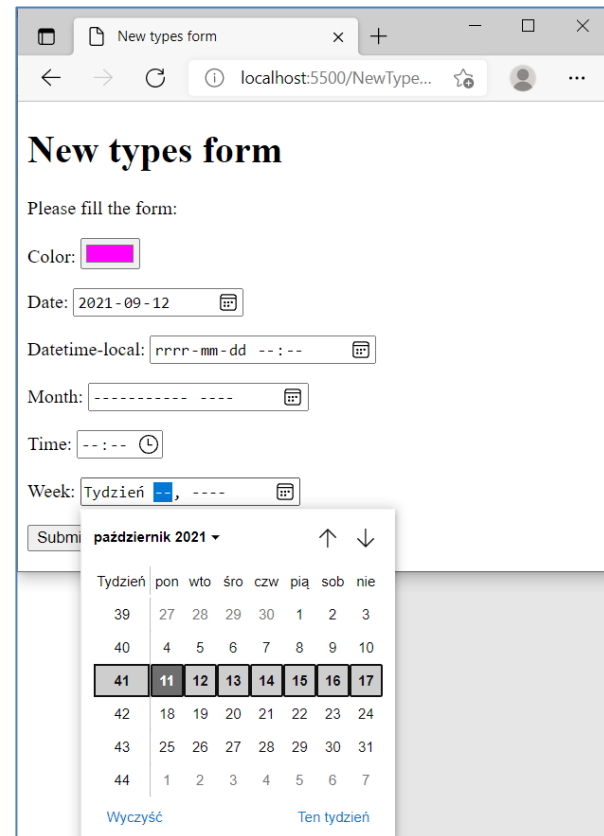
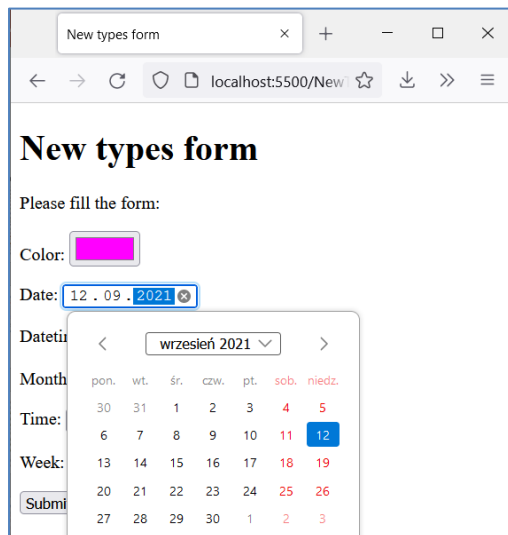
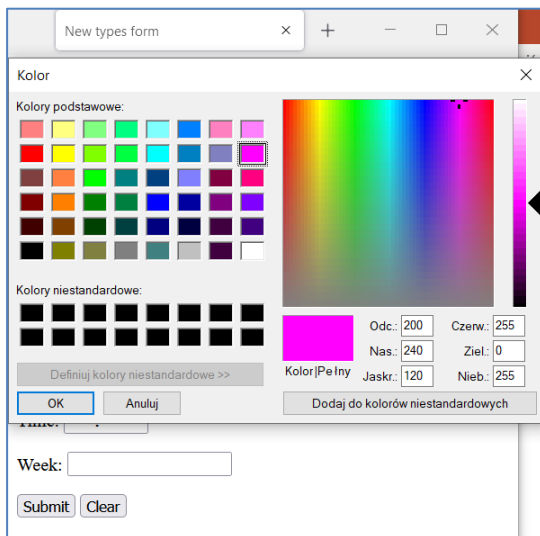
Month:

Time:

Week:

MS Edge 94.0

# Different browser interface - NewTypesForm.html



## New field types `<input>` for format checking and others

- Type `"email"` – checks whether the text is a valid email
- Type `"tel"` – checks whether the phone number is in the correct format
- Type `"url"` – checks whether the given text is the correct URL
- Type `"number"` – for data to be entered, which are an (integer)
  - Attributes `min`, `max`, `step`
- Type `"range"` – slider, with ranges, possible scale, descriptions on the scale, etc.
- Type `"search"` – an icon appears to delete the entered word
- If some data (form controls with these types) does not meet the format, the browser does not allow you to send form data and indicates to the user which data was incorrect
  - No need to write JavaScript code for checking.
- You **always** have to remember that **on the server** you still have to **re-validate** the received data, because someone can send any query with any data using, for example, `curl`.

## Other information related to form fields

- Attribute **autocomplete** – allows you to use user data stored in the browser and entered in other forms under the same **name** attribute. It can have a value "on" or "off". The autocomplete attribute works with the following types: "text", "search", "url", "tel", "email", "password", "range", and "color".
- Example:  

```
<form action="/action_page.php" autocomplete="on">
<input type="email" id="email" name="email" autocomplete
="off">

```
- Element **<datalist>**
  - A set of possible predefined values, typed like a list of elements **<option>**
  - The ability/need to append to another field using the **list** attribute.
  - Typing characters in the field filters in the list, that is, leaving only the values that match the characters you typed in the list.
  - You can still type a different value in the field than the one in **<datalist>**
- Attribute **placeholder** - description (gray by default) inside the form element, which disappears when you type the first character.
- For a field **<input>** there is also **type="file"** for file transfer.
  - if this type of field is used in the form, it must be set to **method="post"** and additionally the attribute **enctype="multipart/form-data"**.
- You can create an image button when you use **type="image"**.
  - Then, in the **src** attribute, we specify the locations of the image file
  - You can use the **width**, **height**, **alt** attributes as for the element **<img>**.



# Example of use - Datalist.html

```
<title>Datalist</title>
</head>
<body>
 <h1>Datalist</h1>
 <form method="post" action="http://localhost:5500/SimpleForm.html">
 <p>
 <label for="flower-choice">Choose a flower:</label>
 <input list="flowers" id="flower-choice" name="FlowerChoice"
 placeholder="flower's name">

 <datalist id="flowers">
 <option value="Brugmansia"/>
 <option value="Rose"/>
 <option value="Tulip"/>
 <option value="Pansy"/>
 <option value="Azalea"/>
 </datalist>
 </p>
 <p>
 <input type="submit" value="Submit">
 <input type="reset" value="Clear">
 </p>
</form>
</body>
</html>
```

## Datalist

Choose a flower:

Submit

Clear

## Datalist

Choose a flower:

Submit

Clear

Brugmansia  
Rose  
Tulip  
Pansy  
Azalea

Datalist

localhost:5500/Data

## Datalist

Choose a flower:

Submit Clear

Brugmansia  
Tulip

## Options in the form – "hidden", readonly, disabled

- `<input type="hidden" name="name" value="value">` defines a hidden input field (invisible to the user) which **will be sent** when you submit the form. A hidden field allows web developers to include data that cannot be seen or modified by users when they submit a form. A hidden field often stores an id of database record that you need to update when you submit a form.
- The **readonly** attribute for `<input>` specifies that the input field is read-only. The value of the read-only input field **will be sent** when you submit the form!  
`<label for="fname">First name:</label><br>`  
`<input type="text" id="fname" name="fname" value="Anna" readonly>`
- Attribute **for** of element `<label>` should be equal to the **id** attribute of element `<input>`, which is to bind them together.
- The **disabled** attribute for `<input>` specifies that the input field should be disabled. The disabled input field is useless and non-clickable. The value of the disabled input field **will not be sent** when you submit the form!  
`<label for="fname">First name:</label><br>`  
`<input type="text" id="fname" name="fname" value="Anna" disabled>`
- The **required** attribute for `<input>` specifies that the input field must be populated before the form can be submitted.  
`<label for="fname">First name:</label><br>`  
`<input type="text" id="fname" name="fname" value="Anna" required>`

## Options in the form – **size**, **maxlength**, **multiple**

- The **size** attribute for `<input>` specifies the **visible width** of the input field expressed in characters. The default size is 20. **Remark!** The **size** attribute works with the following input types: "text", "search", "tel", "url", "email" and "password"  
`<label for="pin">PIN:</label><br>`  
`<input type="text" id="pin" name="pin" size="4">`
- The **maxlength** attribute for `<input>` specifies the maximum number of characters allowed in an input field. **Remark!** When **maxlength** is set, the input field will not accept more than the specified number of characters. However, this attribute does not guarantee that any feedback will be given to the user. So, if you want to warn the user, you need to write JavaScript code.  
`<label for="pin">PIN:</label><br>`  
`<input type="text" id="pin" name="pin"`  
`maxlength="4" size="4">`
- The **multiple** attribute for `<input>` specifies that the user can enter **more than one value** in the input field.
- The **multiple** attribute works with the following types of input: "email" and "file".  
`<label for="pliki">Select files:</label>`  
`<input type="file" id=",,files" name=",,files" multiple>`

# Options in the form

- The **autofocus** attribute for `<input>` specifies that the input field should automatically focus on it when the page loads.  
`<label for="fname">First name:</label><br>  
<input type="text" id="fname" name="fname" value="Anna" autofocus>`
- The **placeholder** attribute for `<input>` specifies a short hint that describes the expected value of the input field (example value or brief description of the expected format). A brief hint is displayed in the input field before the user enters a value.
- The **pattern** attribute specifies the **regular expression** with which the value of the `<input>` element is checked when the form is submitted.
- The **placeholder** attribute works with the following input types: "text", "search", "url", "tel", "email" and "password".  
`<label for="phone">Your phone number:</label>  
 <input type="tel" id="phone" name="phone"  
 placeholder="+48-123-456-789"  
 title="+XX-XXX-XXX-XXX"  
 pattern="+[0-9]{2}-[0-9]{3}-[0-9]{2}-[0-9]{3}">`
- The **step** attribute for `<input>` specifies the proper numeric intervals for the input field. The **step** attribute works with the following input types: "number", "range", "date", "datetime-local", "month", "time" and "week".  
`<label for="points">Points:</label>  
<input type="number" id="points" name="points" step="3">`

## Options in the form

- The **formmethod** attribute for `<input>` defines the HTTP method for sending form data to the action URL. **Remark!** This attribute **overrides** the **method** attribute of element `<form>`. The **formmethod** attribute works with "submit" and "image" type.

```
<form action="send.php" method="get">
 <label for="fname">First name:</label>

 <input type="text" id="fname" name="fname" value="Anna">

 <input type="submit" value="Send using the GET method">
 <input type="submit" formmethod="post" value="Send using the POST method"> </form>
```

- The **formaction** attribute for `<input>` specifies the URL that will process the input when the form is submitted. **Remark!** This attribute overrides the **action** attribute of the element `<form>`.

```
<form action="user.php">
 <label for="fname">Imię:</label>

 <input type="text" id="fname" name="fname" value="Anna" >

 <input type="submit" value="Submit">
 <input type="submit" formaction="admin.php" value="Send as Admin">
</form>
```

## Options in the form

- The **novalidate** attribute is a logical attribute for **<form>**. If present, specifies that **form** data (all inputs) should **not be verified** when submitted.

```
<form action="send.php" novalidate>
 <label for="email">Your e-mail:</label>
 <input type="email" id="email" name="email">

 <input type="submit" value="Send">
</form>
```

- The **formnovalidate** attribute for input specifies **that the element <form> it should not be verified** after submission. Remark! This attribute replaces the **novalidate** attribute of the element **<form>**. The **formnovalidate** attribute works with "submit" type.

```
<form action="send.php">
 <input type="email" id="email" name="email">

 <input type="submit" value="Send">
 <input type="submit" formnovalidate="formnovalidate"
 value="Send without validation"> </form>
```

# Restrictions on data entry

[HTML Input Types \(w3schools.com\)](https://www.w3schools.com/html/html_input_types.asp)

Attribute	Description
checked	Specifies that an input field should be pre-selected when the page loads (for type="checkbox" or type="radio")
disabled	Specifies that an input field should be disabled
max	Specifies the maximum value for an input field
maxlength	Specifies the maximum number of character for an input field
min	Specifies the minimum value for an input field
pattern	Specifies a regular expression to check the input value against
readonly	Specifies that an input field is read only (cannot be changed)
required	Specifies that an input field is required (must be filled out)
size	Specifies the width (in characters) of an input field
step	Specifies the legal number intervals for an input field
value	Specifies the default value for an input field

```
<input type="tel"> phone number
<form>
 <label for="phone_number">Your phone number:</label>
 <input type="tel" id="phone_number" name="phone_number" pattern="[0-9]{3}-[0-9]{2}-[0-9]{3}">
</form>
```

## Tag **<fieldset>**

- Element **<fieldset>** is used to group related data in a form. Element **<legend>** defines a signature for the element **<fieldset>**.
- ```
<form action="/wykonaj.php">
  <fieldset>
    <legend>Personal data:</legend>
    <label for="fname">First name:</label><br>
    <input type="text" id="fname" name="fname" value="Anna" ><br>
    <label for="lastname">Last name:</label><br>
    <input type="text" id="lastname" name="lastname" value="Nowak" ><br><br>
    <input type="submit" value="Submit">
  </fieldset>
</form>
```


Example of use - form3.html

```
<!DOCTYPE html>
<html><head>
</head><body>
  <!-- file: form3.html-->
  <p>This is a text!</p>
  <form action="/wykonaj.php" method="get">
    <fieldset>
      <legend>Personal data:</legend>
      <label for="first_name">First name:</label><br>
      <input type="text" id="first_name" name="first_name" value="Anna" ><br>
      <label for="last_name">Last name:</label><br>
      <input type="text" id="last_name" name="last_name" value="Nowak" ><br><br>
      <input type="submit" value="Submit"> <!--Pierwszy przycisk Submit-->
    </fieldset>
    <fieldset>
      <legend>Your car:</legend>
      <label for="type">car:</label>
      <select id="type" name="type" size="3" multiple>
        <option value="volvo">Volvo</option>
        <option value="saab">Saab</option>
        <option value="fiat" selected>Fiat</option>
        <option value="audi">Audi</option>
      </select>
    </fieldset>
    <input list="program" >
    <datalist id="program">
      <option value="Internet Explorer">
      <option value="Firefox">
      <option value="Chrome">
      <option value="Opera">
      <option value="Safari">
    </datalist><br><br><br><br><br>
    <textarea name="message" rows="5" cols="30"> text 1.</textarea><br> <!--the same item name
is twice/dwukrotnie taka sama nazwa elementu-->
    <textarea name="message" style="width:200px; height:100px;"> text 2. </textarea><br> <!--
the same item name is twice/dwukrotnie taka sama nazwa elementu-->
    <button type="button" onclick="alert('Hello World!')">Click!</button>
    <input type='submit' value='Submit - GET'> <!--Second Submit button - GET method//Drugi
przycisk Submit - metoda GET-->
    <input type="submit" formmethod="post" value="Submit - POST"> <!--Submit - POST method/
Submit - metoda POST-->
    <input type="submit" formaction="admin.php" value="Submit as Admin"> <!--Submit - GET
method/Submit - metoda GET, uruchamianie innej strony-->
  </form>
</body></html>
```

This is a text!

Personal data:

First name:
Anna

Last name:
Nowak

Submit

Your car:

Volvo
Saab
car: Fiat

text 1.

text 2.

Click! Submit - GET Submit - POST Submit as Admin

Example of use - form4.html

```
<!DOCTYPE html>
<html><head>
</head><body>
  <!--file: form4.html-->
<p>Form!</p>
<form action="/wykonaj.php" oninput="x.value=parseInt(a.value)+parseInt(b.value)">
  <fieldset>
    <legend>New form elements:</legend>
    0 <input type="range" id="a" name="a" value="50"> 100 +
    <input type="number" id="b" name="b" value="50"> =
    <output name="x" for="a b"></output>
    <br>
    <label for="ulub_color">New form elements:</label>
    <input type="color" id="ulub_color" name="ulub_color"><br>
    <label for="datemax">Date before 1980-01-01:</label>
    <input type="date" id="datemax" name="datemax" max="1979-12-31"><br>
    <label for="datemin">Date after 2000-01-01:</label>
    <input type="date" id="datemin" name="datemin" min="2000-01-02"><br>
    <label for="birthdaytime">Birthday (date and time):</label>
    <input type="datetime-local" id="birthday_time" name="birthday_time"><br>
    <label for="email">Your e-mail:</label>
    <input type="email" id="email" name="email"><br>
    <label for="file">Select a file:</label>
    <input type="file" id="file" name="file"><br>
    <label for="first_name">First name:</label>
    <input type="text" id="first_name" name="first_name"><br>
    <input type="hidden" id="custId" name="custId" value="1234">
    <input type="submit" value="Submit">
  </fieldset>
</form>
</body></html>
```

Form!

New form elements: 0 100 + =

New form elements:

Date before 1980-01-01:

Date after 2000-01-01:

Birthday (date and time):

Your e-mail:

Select a file: Nie wybrano pliku

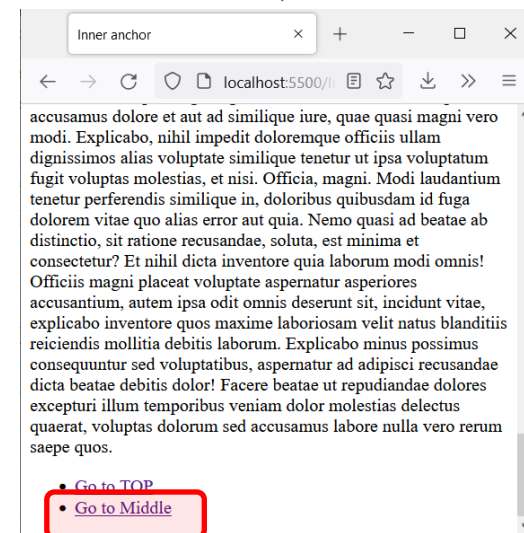
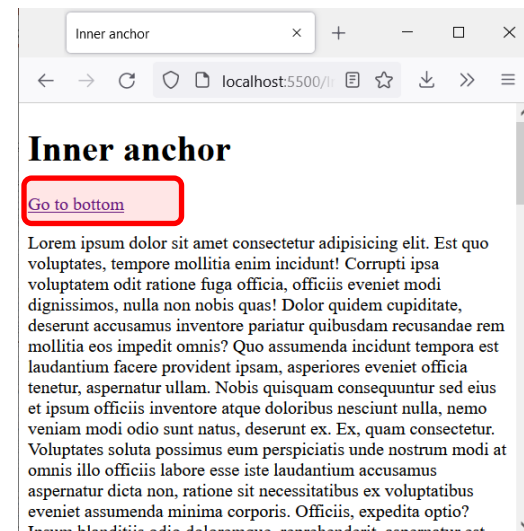
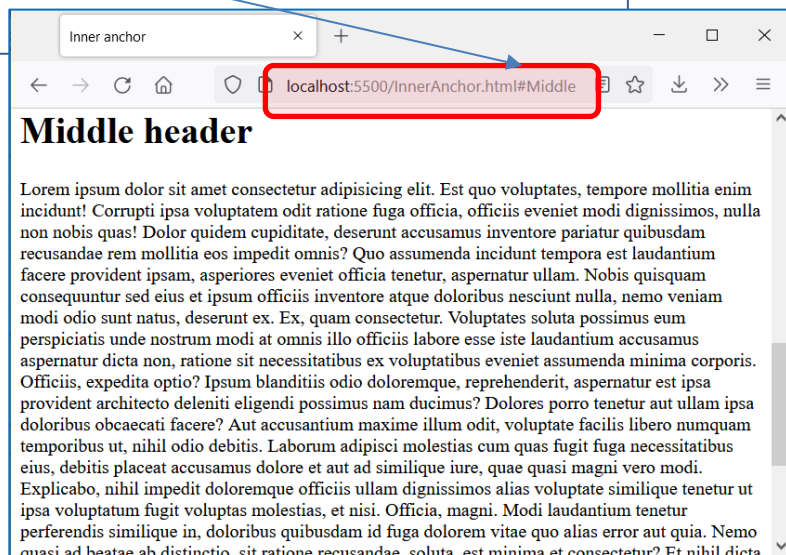
First name:

Internal linking

- The tag `<a>` can be used to link to other sections of the same document by specifying the `id` of another element as `href` attributes.
 - More precisely, in the quotation marks of the value `href` must be the character '#', after which the value `id` of this element is directly typed.
- A segment appears in the URL describing access to this section after the '#' character, e.g.
 - <http://localhost:5500/InnerAnchor.html#Middle>
- Of course, such an address can also be entered from the keyboard or linked from another page.
- If such an identifier on the page does not exist and this part of the URL is incorrect, it is ignored.

Internal linking – example InnerAnchor.html

```
<title>Inner anchor</title>
</head>
<body>
  <h1 id="Top">Inner anchor</h1>
  <p>
    <a href="#Bottom">Go to bottom</a>
  </p>
  <!-- lorem300<TAB> -->
  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Est o
  </p>
  <h1 id="Middle">Middle header</h1>
  <!-- lorem300<TAB> -->
  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Est o
  </p>
  <ul id="Bottom">
    <li><a href="#Top">Go to TOP</a></li>
    <li><a href="#Middle">Go to Middle</a></li>
  </ul>
</body>
</html>
```



External linking

- By default, the linked page will be displayed in the current browser window. To change this, you must specify a different link target.
- The **target** attribute specifies where to open the linked document and can have one of the following values:
 - `"_self"` - Default. Opens the document in the same window/tab where you clicked
 - `"_blank"` - Opens the document in a new window or tab
 - `"_parent"` - Opens the document in the parent frame
 - `"_top"` - Opens the document in the entire part of the window
- An example: `<a href="https://www.pwr.edu.pl/" target="_blank">Politechnika Wrocławska</a>`
- To use an **image** as a link, simply place a tag `<img>` inside the tag `<a>`:
`
`
- To use a **button** as a link, you need to add JavaScript code. JavaScript allows you to specify what happens during specific events, such as clicking a button:
`<button onclick="document.location='default.asp'">HTML Tutorial</button>`

Send an email

- Components of the e-mail, which can be used as part of query string:
 - `subject`
 - `cc` – to whom to send a copy of the message (carbon copy),
 - `bcc` – hidden recipient of a copy of the message (blind carbon copy),
 - `body` - message content.
- Examples:
 - `<a href="mailto:anna@pwr.edu.pl, piotr@pwr.edu.pl">Send e-mail</a>`
 - `<a href="mailto:anna@pwr.edu.pl?subject=email has been sent from the site"> Send e-mail </a>`
 - `<a href="mailto:email@example.com?cc=jan@pwr.edu.pl, piotr@pwr.edu.pl,&bcc= marta@pwr.edu.pl&subject= email has been sent from the site"> Send e-mail </a>`
 - `<a href="mailto:marta@pwr.edu.pl?cc= anna@pwr.edu.pl, marek@pwr.edu.pl,&bcc=jan@pwr.edu.pl&subject=email has been sent from the site&body=Message"> Send e-mail </a>`
- Using links with the `mailto` command is quite easy and helpful. However, this is a form of sharing your e-mail address with other Internet users, in particular those who can use this address to send SPAM.
- This is one of the most common ways spammers obtain email addresses. Often, a good anti-spam program will not help here. Therefore, a better solution is to provide a **contact form**.

Elements of the page structure

- Elements of the logical division of the document structure, creating groups or separate fragments, useful for creating a master page.
 - Element **<header>** - page or section header (not to be confused with elements **<h1>** . . **<h6>**), something like a banner.
 - Element **<main>** - the main substantive part of the page
 - Element **<footer>** - footer of the page / section
- Element **<nav>** - navigation element, page menu. Most often it contains a list (**/**, also nested) of links to the same page, other application pages, external sites. Often inside the **<header>**.
- Element **<article>** - logically separated text article
- Element **<section>** - Element section
- Element **<aside>** - content beyond the main information of the page (digressions)
- All these elements are particularly useful using CSS to visually separate the header, footer, menus, articles, etc., differently respond to click-on events, etc.
 - Without styling, they are no different from, for example, an element **<p>** etc.

Example LogicalDivision.html

```
<title>Logical division</title>
</head>
<body>
  <header>
    <nav>
      <ul>
        <a href="SimpleForm.html">Simple</a>
        <a href="ComplexForm.html">Complex</a>
      </ul>
    </nav>
  </header>
  <main>
    <h1>Logical division - main</h1>
    <article>
      <header>
        The most important article
      </header>
      Lorem ipsum dolor sit amet consectetur adipisicing elit.
    </article>
  </main>
  <footer>
    &copy; Dariusz Konieczny
  </footer>
</body>
</html>
```

[Simple](#) [Complex](#)

Logical division - main

The most important article

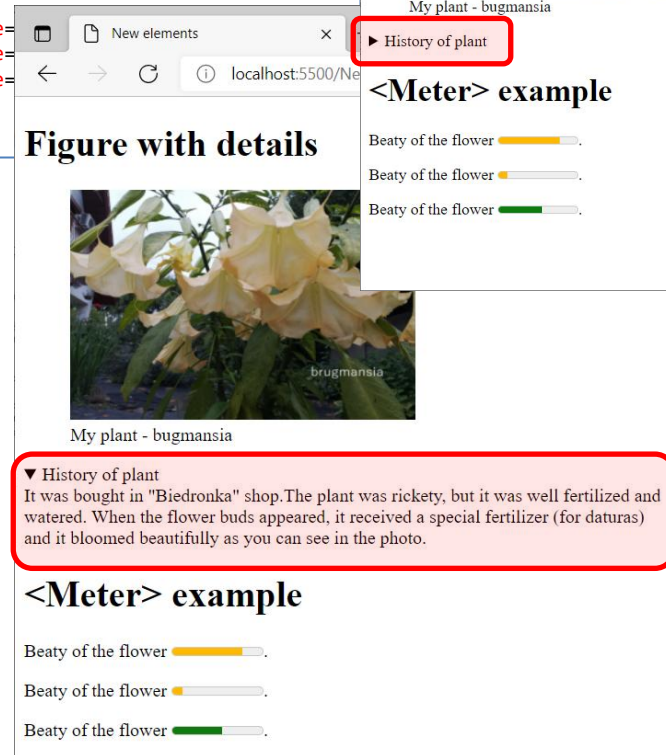
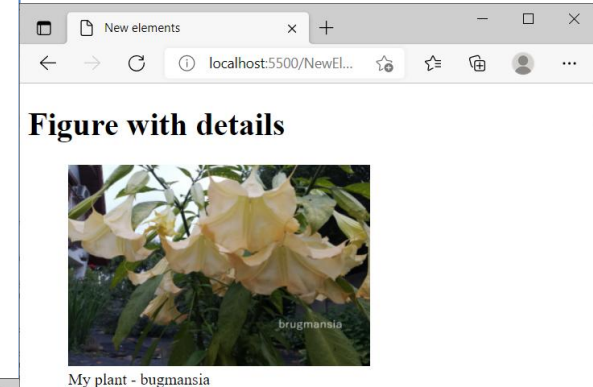
Lorem ipsum dolor sit amet consectetur adipisicing elit. Facere, doloremque laudantium officiis, adipisci molestiae, expedita nesciunt officia nobis. Arc Quasi odit, rerum suscipit ipsum pariatur quibusdam repellat quaerat nisi n aliquid recusandae rerum quasi vel rem quibusdam nobis. Ut mollitia dolor nam sint odio, possimus repellat a ducimus, accusamus iure odit quas error quidem impedit dicta ab sequi, atque modi repudiandae eveniet, veniam eri Obcaecati eum saepe ullam voluptate quam libero, nisi incidunt facere, vita © Dariusz Konieczny

Other new elements

- Element **<figure>** - the element most often surrounding **** but also an arbitrarily constructed illustration. Allows you to add a title/caption to this.
- Element **<figcaption>** - most often inside the element **<figure>**, the title of the illustration.
 - Standard above the illustration.
- Element **<details>** - Creates a expandable element with a detailed description. Usually marked as a triangle pointing to the right. When clicked, the triangle faces downwards and shows the content contained in the **<details>**.
- Element **<summary>** - inside element **<details>**. It is the label of this element, that is, it appears to the right of the triangle.
- Using the above elements makes them easier to access for people with disabilities and allows you to add a description with more details.
- Element **<meter>** - a graphic element in the type of progress bar.

Example NewElements.html

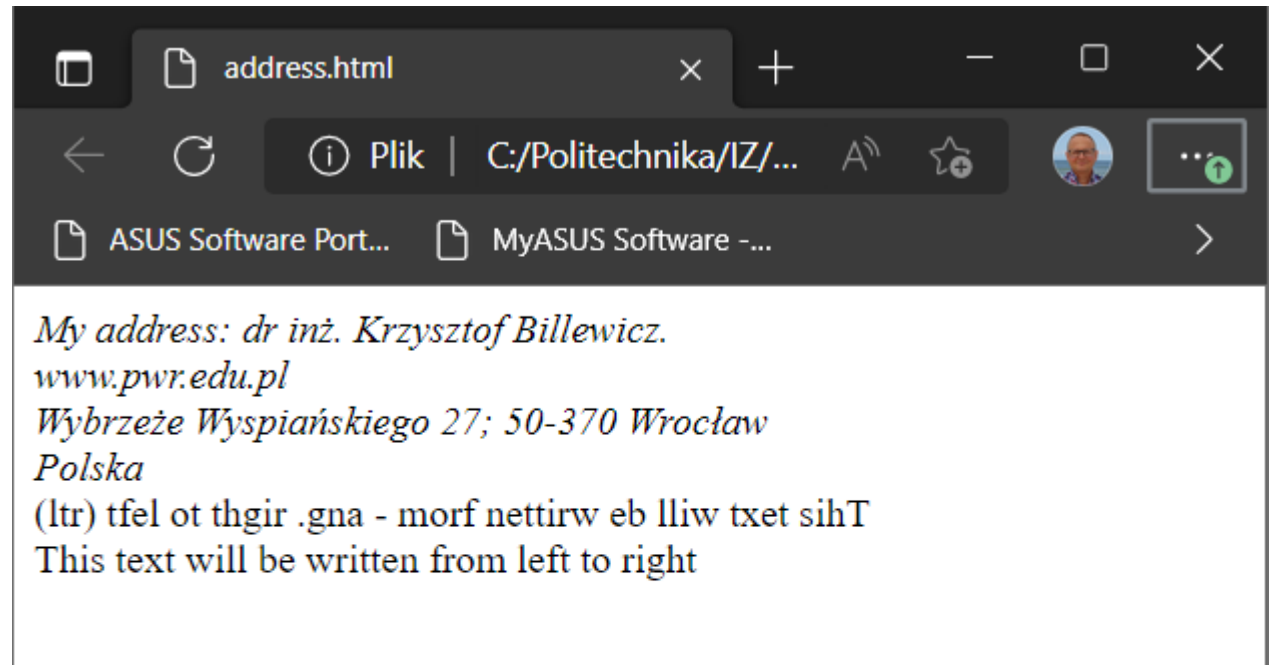
```
<title>New elements</title>
</head>
<body>
  <h1>Figure with details</h1>
  <figure>
    
    <figcaption>My plant - bugmansia</figcaption>
  </figure>
  <details>
    <summary>History of plant</summary>
    It was bought in "Biedronka" shop. The plant was rickety, but it was well
    fertilized and watered. When the flower buds appeared, it received
    a special fertilizer (for daturas) and it bloomed beautifully
    as you can see in the photo.
  </details>
  <h1>&lt;Meter&gt; example</h1>
  <p>Beaty of the flower <meter min="10" max="100" low="30" high="75" value=
  <p>Beaty of the flower <meter min="10" max="100" low="30" high="75" value=
  <p>Beaty of the flower <meter min="10" max="100" low="30" high="75" value=
</body>
</html>
```



Citations and citation elements

- An HTML element `<blockquote>` defines a section that is cited from another source. Browsers usually indent items.
`<p>`This is a quote from the pages of the Wrocław University of Technology:`</p>`
`<blockquote cite="https://pwr.edu.pl/uczelnia/aktualnosci/zasady-funkcjonowania-uczelni-w-roku-akademickim-2020-201-11710.html">`
On January 20, the Rector of the Wrocław University of Technology issued a statement on the principles of work on...`</blockquote>`
- An HTML tag `<q>` defines a short citation. Browsers usually put quotation marks around a quote.
`<p>`About our university: `<q>`Wrocław University of Technology was established in 1945. `<q></p>`
- An HTML tag `<abbr>` defines a shortcut or an acronym such as "HTML", "CSS", "Mr.", "Dr.", "ASAP", "ATM".
`<p>`University `<abbr title="Wroclaw University of Technology">`WUT`</abbr>` was established in 1945.`</p>`
- The HTML tag `<cite>` defines the title of a creative work (e.g., a book, poem, song, film, painting, sculpture, etc.). Text in an element is usually rendered in *italics*.
`<p><cite>`Pan Tadeusz`</cite>` Adam Mickiewicz's was published in 1834 in Paris.`</p>`
- The HTML tag `<bdo>` is used to replace the current direction of the text:
`<bdo dir="rtl">`This text will be written from right to left (rtl)`</bdo>`

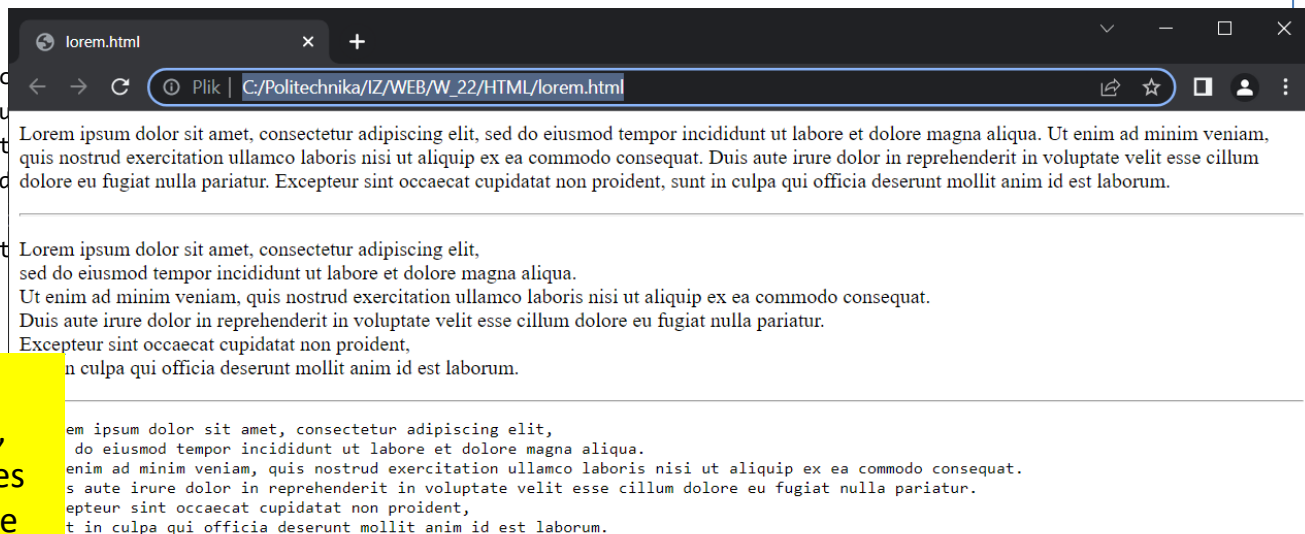
Example of text formatting - address.html



```
<!DOCTYPE html> <head> </head> <body>
  <!--File: adres.html-->
  <p>
    <address>My address: dr inż. Krzysztof Billewicz.<br>
      www.pwr.edu.pl<br>
      Wybrzeże Wyspiańskiego 27; 50-370 Wrocław<br>
      Polska</address>
    <bdo dir="rtl">This text will be written from - ang. right to left (rtl)</bdo><br>
    <bdo dir="ltr">This text will be written from left to right</bdo>
  </p>
</body></html>
```

Example of text formatting - lorem.html

```
<!DOCTYPE html> <head> </head> <body>
  <!-- file: lorem.html-->
  <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit,
    sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
    Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
    Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.
    Excepteur sint occaecat cupidatat non proident,
    sunt in culpa qui officia deserunt mollit anim id est laborum.</p>
  <hr>
  <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, <br>
    sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. <br>
    Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. <br>
    Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. <br>
    Excepteur sint occaecat cupidatat non proident, <br>
    sunt in culpa qui officia deserunt mollit anim id est laborum.</p>
  <hr>
  <pre>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit,
    sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
    Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
    Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.
    Excepteur sint occaecat cupidatat non proident,
    sunt in culpa qui officia deserunt mollit anim id est laborum.
  </pre>
</body> </html>
```



Text in an element `<pre>` is displayed in a fixed-width font, and the text retains both spaces and line breaks. The text will be displayed exactly as it is written in the HTML source code.

Element `<iframe>`

- An `<iframe>` tag is used to display a web page on another web page. Element `<iframe>` specifies a built-in frame. Use a built-in frame to embed another document in the current HTML document.
- Remark! It is a good practice to include the `title` attribute for the `<iframe>`. This is used by screen readers to read the contents of an `<iframe>`.
- You can specify the size of an `<iframe>` by defining attributes `height` and `width`. Height and width are specified in pixels by default:

```
<iframe src="ex_iframe.htm" height="300" width="300" title="An example of using iframe"></iframe>
```

- You can also add a `style` attribute and use CSS, and define the size of the element by specifying the height and width properties. By default, an `<iframe>` has a border around it.
- To remove a border, add an attribute and use the CSS property like below:

```
<iframe src="table.html" style="border:none;" title="An example of using iframe"></iframe>
```

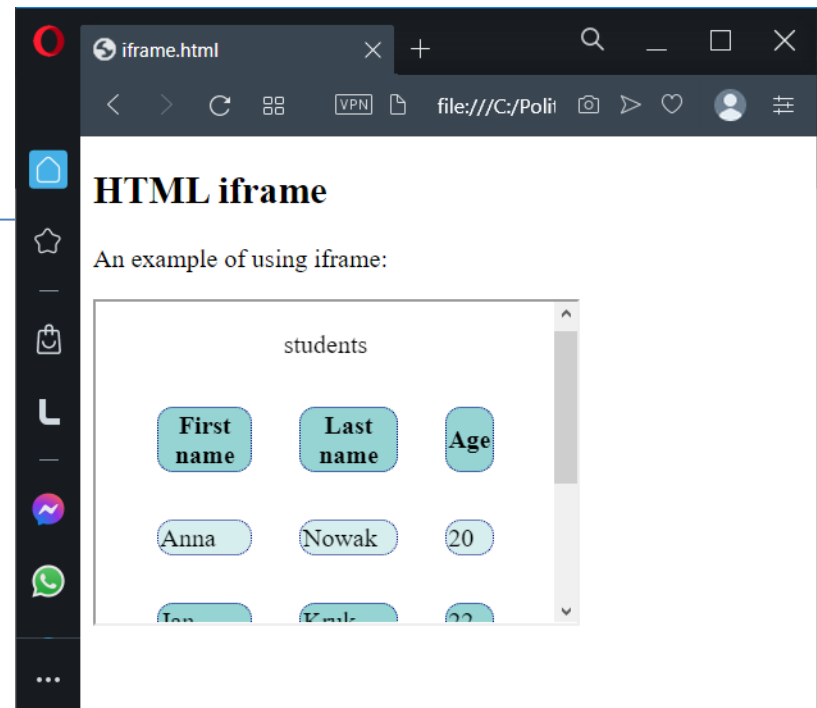
- You can also use CSS to change the size, style, and color of the border of an `<iframe>`:

```
<iframe src="table.html" style="border:2px solid red;" title="An example of using iframe"></iframe>
```

Example – iframe.html

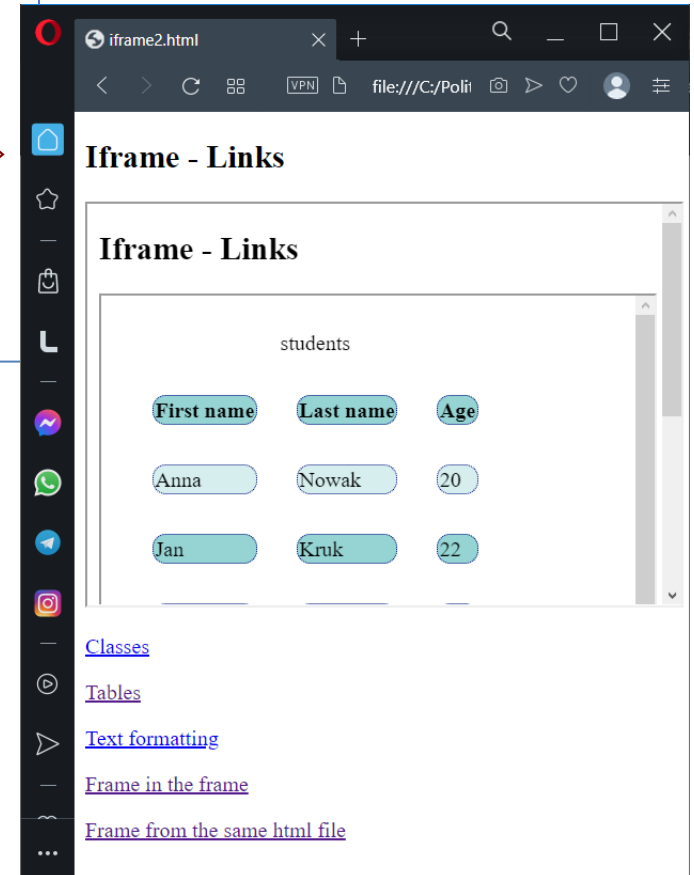
```
<!DOCTYPE html>
<html><body>
<!--file: iframe.html-->
<h2>HTML iframe</h2>
<p> An example of using iframe: </p>

<iframe src="table.html" style="height:200px;width:300px" title="An example of
using iframe"></iframe>
</body></html>
```



Przykład – iframe2.html

```
<!DOCTYPE html>
<html><body>
<!--file: iframe2.html-->
<h2>Iframe - Links</h2>
<iframe src="table.html" name="iframe_a" height="300px" width="100%"
title="An example of using iframe"></iframe>
<p><a href="class2.html" target="iframe_a">Classes</a></p>
<p><a href="table.html" target="iframe_a">Tables</a></p>
<p><a href="colors.html" target="iframe_a">Text formatting</a></p>
<p><a href="iframe.html" target="iframe_a">Frame in the frame</a></p>
<p><a href="iframe2.html" target="iframe_a">Frame from the same html
file</a></p>
</body>
</html>
```

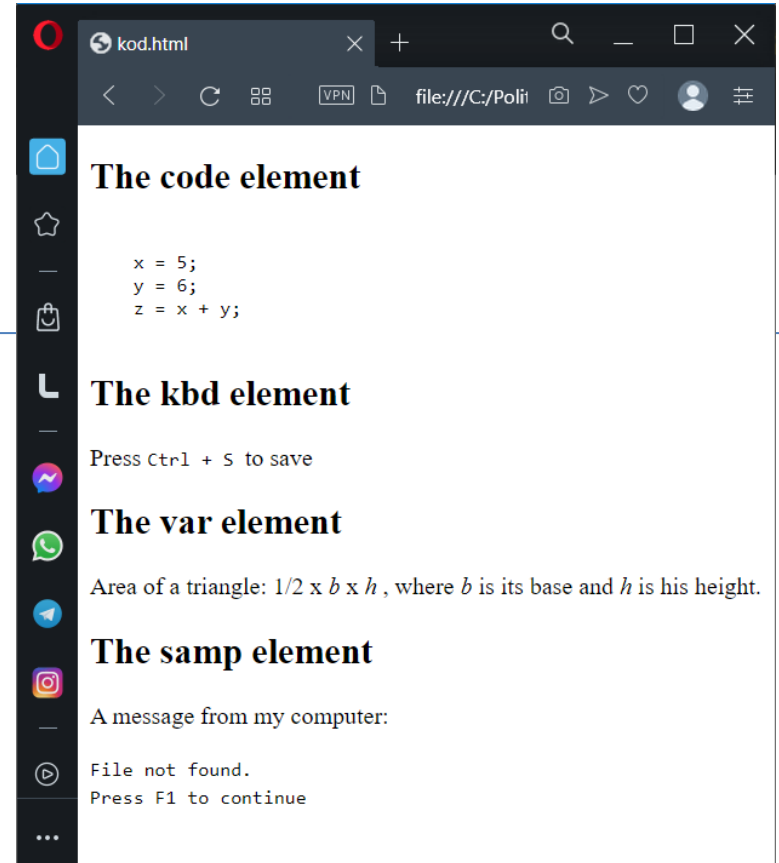


HTML computer code elements

- HTML contains several elements for defining user input and computer code. This code is not executed here!!!
`<code>`
x = 5;
y = 6;
z = x + y;
`</code>`
- Tag `<code>` does not retain extra spaces and line breaks. To fix this, you can place the `<code>` element inside the `<pre>` element.
`<pre><code>`
x = 5;
y = 6;
z = x + y;
`</code></pre>`
- Tag `<kbd>` is used to define keyboard input. Content inside is displayed in the browser's default monospace font.
`<p> Press <kbd> Ctrl + S </kbd> to save </p>`
- Tag `<samp>` is used to define sample output from a computer program. Content inside is displayed in the browser's default monospace font.
`<p> A message from my computer: </p>`
`<p> <samp> File not found. <br> Press F1 to continue </samp> </p>`
- Tag `<var>` is used to define a variable in programming or in a mathematical expression. Content inside is usually italicized.
`<p> Area of a triangle: 1/2 x <var> b </var> x <var> h </var>, where <var> b </var> is its base and <var> h </var> is his height. </p>`

Example of text formatting - code.html

```
<!DOCTYPE html>
<html><body>
  <!--File: code.html-->
  <h2> The code element </h2>
  <pre> <code>
x = 5;
y = 6;
z = x + y;
  </code> </pre>
  <h2> The kbd element </h2>
  <p> Press <kbd> Ctrl + S </kbd> to save </p>
  <h2> The var element </h2>
  <p> Area of a triangle: 1/2 x <var> b </var> x <var> h </var>, where <var> b </var> is its base
and <var> h </var> is his height. </p>
  <h2> The samp element </h2>
  <p> A message from my computer: </p>
  <p> <samp> File not found. <br> Press F1 to continue </samp> </p>
</body></html>
```



Special characters

- Certain characters within HTML5 have a special meaning, e.g. '<', '?', '&', quotation marks, etc.
 - Therefore, they cannot be directly a component of the data (attributes, query string, etc.)
- The way to represent such data are special characters (in the format `&code;`) where the `code` is written as:
 - abbreviation of the word
 - number:
 - decimal
 - hexadecimal

Symbol	Description	Notation
&	ampersand	&
'	apostrophe	'
>	greater-than	>
<	less-than	<
"	quote	"

Special characters

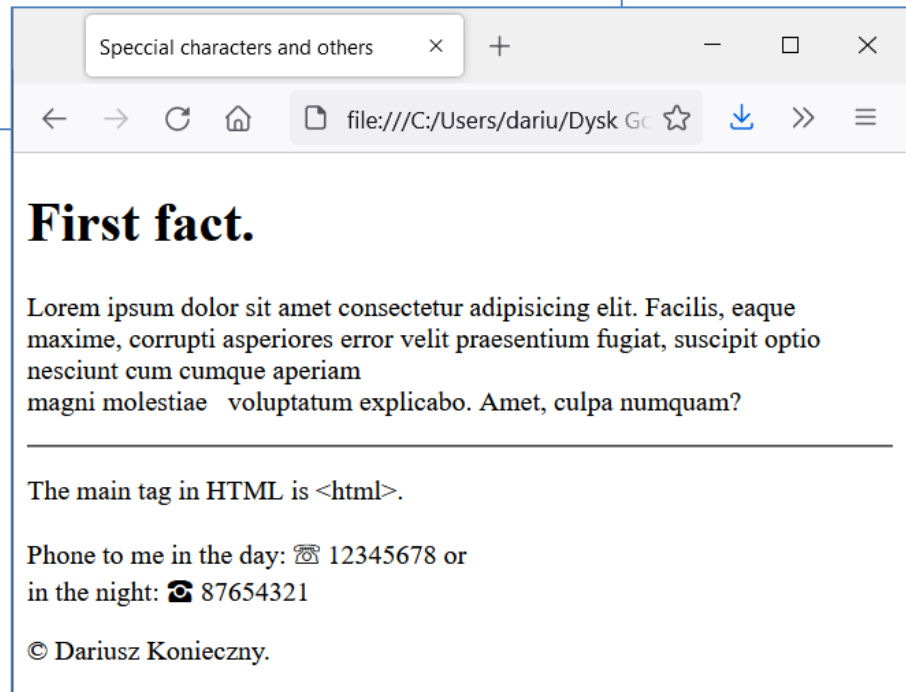
- Also, other characters that are absent from the keyboard should not be artificially generated (the general rule is that in HTML there can only be symbols that can be reached from the keyboard)
- For them, the format `&code;` is also used
- Code tables:
 - <https://dev.w3.org/html5/html-author/charref>
 - https://www.w3schools.com/html/html_symbols.asp
 - <https://www.toptal.com/designers/htmlarrows/>
 - <https://ascii.cl/htmlcodes.htm>

Symbol	Description	Notation
©	copyright	©
<	less-than	<
	phone	☎
	phone	☏
®	Registered trademark	®
	Non-braking space	
™	trademark	™

Example SpecCharacters.html

- Code start deleted (space-saving)

```
<title>Speccial characters and others</title>
</head>
<body>
  <h1>First fact.</h1>
  <p>
    Lorem ipsum dolor          sit amet consectetur adipisicing elit.
    Facilis, eaque maxime, corrupti asperiores error velit
    praesentium fugiat, suscipit optio nesciunt cum cumque aperiam
    magni&nbsp;molestiae&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;voluptatum&nbsp;explicabo.&nbsp;Amet,&nbsp;culpa numquam?
  </p>
  <hr>
  <p>The main tag in HTML is &lt;html&gt;.</p>
  <p>
    Phone to me in the day: &#9743; 12345678 or <br> in the night: &phone; 87654321
  </p>
  <p>&copy; Dariusz Konieczny.</p>
</body>
</html>
```



Registered marks - to be replaced by character entities

Character	Description	Entity's name	Entity number
	non-separating space	 	
<	less than	<	<
>	greater than	>	>
&	ampersand	&	&
"	double quotation marks	"	"
'	single quotation mark (apostrophe)	'	'
¢	cent	¢	¢
£	pound	£	£
¥	yen	¥	¥
€	euro	€	€
©	copyright	©	©
®	registered trademark	®	®

- More characters supported in HTML:
https://www.w3schools.com/html/html_symbols.asp

Some mathematical symbols

Character	Number	Entity	Description
$\forall$	∀	∀	FOR ALL
∂	∂	∂	PARTIAL DIFFERENTIAL
$\exists$	∃	∃	THERE EXISTS
$\emptyset$	∅	∅	EMPTY SETS
∇	∇	∇	NABLA
$\in$	∈	∈	ELEMENT OF
$\notin$	∉	∉	NOT AN ELEMENT OF
$\ni$	∋	∋	CONTAINS AS MEMBER
$\prod$	∏	∏	N-ARY PRODUCT
$\sum$	∑	∑	N-ARY SUMMATION

- Full set of mathematical characters supported in HTML:
https://www.w3schools.com/charsets/ref_utf_math.asp

Emoticons

- Emoticons look like images or icons, but they are not images. These are letters (characters) from the UTF-8 (Unicode) character set.
- UTF-8 includes almost all characters and symbols in the world.
- For an HTML page to display correctly, your web browser must know the character set used on the page.
- This is specified in the **<meta>** tag:
<meta charset="UTF-8">
 - if not specified, UTF-8 is the default character set in HTML.
- If you want any of these emoticons to appear on the page in HTML, you can use the decimal or hexadecimal reference in the corresponding table.
- Full set of HTML supported emoticons:
https://www.w3schools.com/charsets/ref_emoji.asp

Some emoticons in UTF-8

Emoticons	Values
	🗻
	🗼
	🗽
	🗾
	🗿
	😀
	😁
	😂
	😃
	😄
	😅

URL encoding

- URLs can only be sent over the Internet using the ASCII character set. If the URL contains non-ASCII characters, the URL must be converted.
- URL encoding converts non-ASCII characters to a format that can be transmitted over the Internet.
- URL encoding replaces non-ASCII characters with a "%" followed by hexadecimal digits. URLs cannot contain spaces. URL encoding usually replaces a space with a plus sign (+) or %20
- Complete information on all URL encodings:
https://www.w3schools.com/tags/ref_urlencode.asp

Element `<canvas>`

- The element `<canvas>` is used to draw graphics on a web page.
 - By default, an item `<canvas>` has no border or content.
- The element `<canvas>` is used to draw graphics, on the fly, via JavaScript.
- An element `<canvas>` is just a container for graphics. You need to use JavaScript to actually draw the graphic.
- Tag `<canvas>` has several methods to draw paths, fields, circles, text, and add images.
- **Remark! Always** specify the `id` attribute (which you need to reference in the script) and the `width` and `height` attributes that define the size of the canvas. Those elements are indispensable. To add a border, use the `styles` attribute.
- Canvas can draw colored text, with or without animation.
- Canvas has great features for graphical presentation of data with images of charts and graphs.
- Canvas objects can move. Everything is possible: from simple bouncing balls to complex animations.
- More about the canvas is on the page:
https://www.w3schools.com/graphics/canvas_intro.asp
<https://www.html5canvastutorials.com/tutorials/html5-canvas-element/>

Canvas - examples 1/2

```
<canvas id="myCanvas" width="200" height="100" style="border:1px solid #000000;">
</canvas>
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.font = "30px Arial";
ctx.fillText("Hello World", 10, 50);
</script>
```

```
<canvas id="myCanvas" width="200" height="100" style="border:1px solid #000000;">
</canvas>
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.font = "30px Arial";
ctx.strokeText("Hello World", 10, 50);
</script>
```

Canvas - examples 2/2

```
<canvas id="myCanvas" width="200" height="100" style="border:1px solid #000000;">
</canvas>
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.moveTo(0, 0);
ctx.lineTo(200, 100);
ctx.stroke();
</script>
```

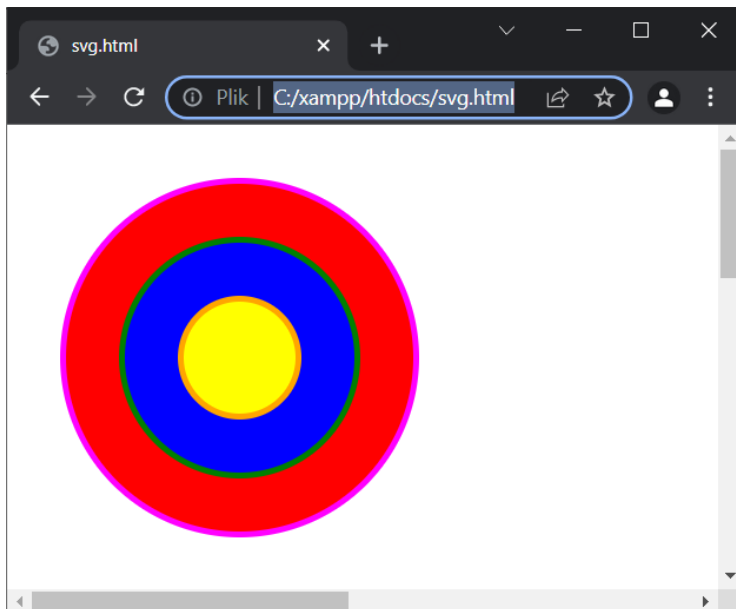
```
<canvas id="myCanvas" width="200" height="100" style="border:1px solid #000000;">
</canvas>
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.beginPath();
ctx.arc(95, 50, 40, 0, 2 * Math.PI);
ctx.stroke();
</script>
```

Canvas - line

- To draw a line using HTML5 Canvas, we can use the methods `beginPath()`, `moveTo()`, `lineTo()` and `stroke()`.
- First, we can use the `beginPath()` method to declare that we are going to draw a new path. Then we can use the `moveTo()` method to position the context point (i.e., the drawing cursor), and then use the `lineTo()` method to draw a straight line from the starting position to the new position. Finally, to make the line visible, we can apply the stroke to the line using `stroke()`. Unless otherwise specified, the stroke color is black by default.
- The `lineWidth` property of the `<canvas>` context specifies the line width in px. The `lineWidth` property must be set before calling the `stroke()` method.

Element `<svg>` 1/2

- SVG stands for Scalable Vector Graphics.
 - SVG is used to define graphics for the Web.
- An element `<svg>` is a container for SVG graphics. SVG has several methods for drawing paths, fields, circles, text, and graphic images.
- More about `<svg>` is on the website:
https://www.w3schools.com/graphics/svg_intro.asp



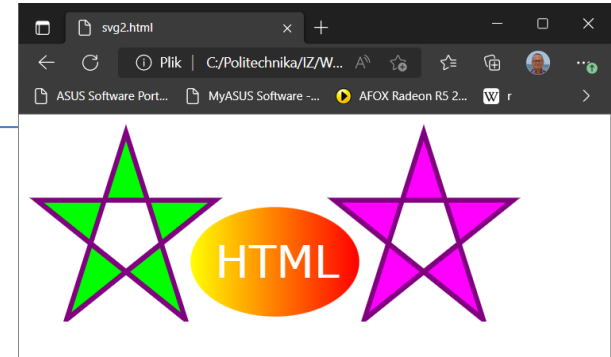
```
<!DOCTYPE html>
<html>
<body>
<!--File: svg.html-->
<svg width="1000" height="1000">
  <circle cx="150" cy="150" r="120"
    stroke="magenta" stroke-width="4" fill="red" />
  <circle cx="150" cy="150" r="80"
    stroke="green" stroke-width="4" fill="blue" />
  <circle cx="150" cy="150" r="40"
    stroke="orange" stroke-width="4" fill="yellow" />
  Sorry, your browser does not support inline SVG.
</svg>
</body>
</html>
```

Element `<svg>` 2/2

```
<!DOCTYPE html>
<html><body>
  <!--file: svg2.htm-->
  <svg width="500" height="200">
    <polygon points="100,10 40,198 190,78 10,78 160,198"
      style="fill:lime;stroke:purple;stroke-width:5;fill-rule:evenodd;" />

    <polygon points="400,10 340,198 490,78 310,78 460,198"
      style="fill:magenta;stroke:purple;stroke-width:5;fill-rule:evenodd;" />

    <defs>
      <linearGradient id="grad1" x1="0%" y1="0%" x2="100%" y2="0%">
        <stop offset="0%" style="stop-color:rgb(255,255,0);stop-opacity:1" />
        <stop offset="100%" style="stop-color:rgb(255,0,0);stop-opacity:1" />
      </linearGradient>
    </defs>
    <ellipse cx="250" cy="140" rx="85" ry="55" fill="url(#grad1)" />
    <text fill="#ffffff" font-size="45" font-family="Verdana" x="190" y="156">HTML</text>
    Sorry, your browser does not support inline SVG.
  </svg>
</body></html>
```



Multimedia

- Multimedia on the Internet is sound, music, movies and animations.
- Multimedia is available in many different formats. It can be almost anything you can hear or see, like images, music, sound, videos, recordings, animations and more.
- Websites often contain multimedia elements of various types and formats.
 - The first web browsers supported text only, limited to a single font in one color.
 - Later, browsers with support for colors, fonts, images and multimedia appeared.
- Media elements (such as audio or video) are stored in media files.
- The most common way to detect a file type is to look at the file extension. Media files have formats and various extensions, such as `.wav`, `.mp3`, `.mp4`, `.mpg`, `.wmv`, and `.avi`.

Media file formats

Format	Extension	Description
MPEG	.mpg .mpeg	MPEG. Developed by the Moving Pictures Expert Group. The first popular video format on the web. Not supported anymore in HTML.
AVI	.avi	AVI (Audio Video Interleave). Developed by Microsoft. Commonly used in video cameras and TV hardware. Plays well on Windows computers, but not in web browsers.
WMV	.wmv	WMV (Windows Media Video). Developed by Microsoft. Commonly used in video cameras and TV hardware. Plays well on Windows computers, but not in web browsers.
QuickTime	.mov	QuickTime. Developed by Apple. Commonly used in video cameras and TV hardware. Plays well on Apple computers, but not in web browsers.
RealVideo	.rm .ram	RealVideo. Developed by Real Media to allow video streaming with low bandwidths. Does not play in web browsers.
Flash	.swf .flv	Flash. Developed by Macromedia. Often requires an extra component (plug-in) to play in web browsers.
Ogg	.ogg	Theora Ogg. Developed by the Xiph.Org Foundation. Supported by HTML.
WebM	.webm	WebM. Developed by Mozilla, Opera, Adobe, and Google. Supported by HTML.
MPEG-4 or MP4	.mp4	MP4. Developed by the Moving Pictures Expert Group. Commonly used in video cameras and TV hardware. Supported by all browsers and recommended by YouTube.

- **Source:** https://www.w3schools.com/html/html_media.asp
- **Note:** Only MP4, WebM, and Ogg video are supported by the HTML standard.

Video in HTML

- The element `<video>` is used to display the video on the web page.
- The `controls` attribute adds video controls such as playback, pause, and volume.
- It's always a good idea to include the width and height attributes. If the `height` and `width` are not set, the page may flicker when the video loads.
- The element `<source>` allows you to specify alternative video files from which the browser can choose. The browser will use the first recognized format.
- Text between tags `<video>` and `</video>` will only be displayed in browsers that do not support `<video>` element.
- To automatically start a video, use the attribute `autoplay`.
- **Remark!** Browsers that use the Chromium codebase do not allow `autoplay` in most cases. However, muted autoplay is always allowed.
 - Add `muted` after `autoplay` to allow the video to play automatically (but muted).
- Example:

```
<video width="320" height="240" autoplay muted>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
Your browser does not support the video tag.
</video>
```
- The HTML DOM defines the methods, properties, and events for the `<video>`.
 - This allows you to load, play and pause videos, as well as set the duration and volume.
- There are also DOM events that can notify you when a video starts playing, is paused, etc.
- More at: https://www.w3schools.com/html/html5_video.asp

Sound in HTML

- The element `<audio>` is used to play an audio file on a web page.
- The `controls` attribute adds audio controls such as playback, pause, and volume.
- The element `<source>` allows you to specify alternate audio files from which the browser can choose. The browser will use the first recognized format.
- Text between tags `<audio>` and `</audio>` will only be displayed in browsers that do not support this element `<audio>`.
- To automatically run the audio file, use the attribute: `autoplay`.
- **Remark!** Browsers that use the Chromium codebase do not allow autoplay in most cases. However, `muted` autoplay is always allowed.
 - Add `muted` after `autoplay` so that the audio file can start playing automatically (but muted)

- Example:

```
<audio controls autoplay muted>
```

```
  <source src="horse.ogg" type="audio/ogg">
```

```
  <source src="horse.mp3" type="audio/mpeg">
```

```
Your browser does not support the audio element.
```

```
</audio>
```

- Audio file formats used in HTML: MP3 (audio/mpeg), OGG (audio/ogg), wav (audio/wav).

Elements `<object>` and `<embed>`

- The item `<object>` is supported by all browsers.
- An element `<object>` defines an object embedded in an HTML document. It is designed to embed plug-ins (such as Java applets, PDF readers, and Flash players) into web pages, but it can also be used to attach HTML to HTML:
`<object width="100%" height="500px" data="snippet.html">`
`</object>`
`<object data="audi.jpeg"></object>`
- The item `<embed>` is supported in all major browsers.
- The element `<embed>` also defines an object embedded in the HTML document.
`<embed width="100%" height="500px" src="snippet.html">`
- Web browsers have long supported `<embed>` element . However, it was not part of the HTML specification before HTML5. The item `<embed>` does not have a closing tag. Must not contain `alt` text. The element `<embed>` can also be used to attach HTML to HTML.
- Example:
`<embed width="100%" height="500px" src="snippet.html">`
- Suggestion
 - To display a picture, it is better to use the `<img>` tag.
 - To display HTML, it is better to use the `<iframe>` tag.
 - To display video or audio, it is better to use the `<video>` and `<audio>` tags.

Movies

- One of the easiest ways to play videos in HTML is to use YouTube service.
- YouTube will display an identifier (e.g. tgbNymZ7vqY) when you save (or play) a video.
- When using an element `<iframe>`, all you have to do is set the value of the `src` attribute to the URL of the video.
 - Of course, you should also set the `width` and `height` attributes
- You can also allow your video to start playing automatically when a user visits the page by adding `"autoplay=1"` to the YouTube URL as a query string.
 - However, automatically launching a video is annoying for visitors!

- Example:

```
<iframe width="420" height="315"
src="https://www.youtube.com/embed/tgbNymZ7vqY">
</iframe>
```

- Remark! Browsers that use the Chromium codebase do not allow autoplay in most cases. However, muted autoplay is always allowed.

- Add `"mute=1"` after `"autoplay=1"` to allow the video to play automatically (but muted)

- Example:

```
<iframe width="420" height="315"
src="https://www.youtube.com/embed/tgbNymZ7vqY?autoplay=1&m
ute=1">
</iframe>
```

- You can also disable controls for the player by adding `"controls=0"`.

- Example:

```
<iframe width="420" height="315"
src="https://www.youtube.com/embed/tgbNymZ7vqY?controls=0">
</iframe>
```

Best Practices 1/6

- Consistent, clean, and organized HTML makes it easier for others to read and understand the code. Here are some tips and tricks for creating good HTML.
- Always declare the document type as the first line of the document. The valid document type for HTML is:
`<!DOCTYPE html>`
- In HTML, you don't have to close all elements (e.g. `<p>`). However, we recommend that you **close all HTML elements** as follows::

```
<section>  
  <p>This is a paragraph.</p>  
</section>
```
- HTML allows you to blend uppercase and lowercase letters in **attribute names**.
- However, we recommend that you use attribute names in **lowercase** because:
 - Mixing uppercase and lowercase letters looks bad
 - Programmers usually use lowercase names
 - Lowercase letters look cleaner
 - Lowercase letters are easier to write
- HTML allows you to mix uppercase and lowercase letters in **element names**.
- However, we recommend using **lowercase** item names because:
 - Mixing uppercase and lowercase letters looks bad
 - Programmers usually use lowercase names
 - Lowercase letters look cleaner
 - Lowercase letters are easier to write

Best Practices 2/6

- Always enclose attribute values in quotation marks. You **MUST** use quotation marks if the value contains spaces. Example:
`<table class="standardowa">`
- Always specify the **alt** attribute of images. This attribute is important if for some reason the image cannot be displayed.
- In addition, always define the **width** and **height** of images. This reduces flickering because the browser can reserve space for the image before loading.
- HTML allows spaces around equal signs. But without spaces, it is easier to read and better groups attribute-value pairs.
Good `<link rel="stylesheet" href="styles.css">`
Bad `<link rel = "stylesheet" href = "styles.css">`
- Try to avoid too long lines of code. When using the HTML editor, it is **NOT** convenient to scroll right and left to read the HTML code.
- Don't add blank rows, spaces, or indents without a reason.
- For readability, add blank lines to separate large or logical blocks of code.
- For readability, add two spaces of indentation. Do **not use** Tab.

Best Practices 3/6

- Element `<title>` is required in HTML.
- The content of the page title is very important for search engine optimization (SEO)! The page title is used by search engine algorithms to decide the order in which pages appear in search results.
- Element `<title>`:
 - defines the title in the browser toolbar
 - provides the title of the page when you add it to your favorites
 - displays the page title in the search results
- So try to make the title as accurate and meaningful as possible.

Best Practices 4/6

- Always place the `lang` attribute inside the tag `<html>` to declare the language of the Web page. This is to help search engines and browsers.

```
<!DOCTYPE html>
<html lang="en-us">
```

- To ensure proper interpretation and correct indexing in search engines, both language and character encoding should be defined as early as possible in the HTML document:

- ```
<meta charset="charset">
<meta charset="UTF-8">
```

- Short comments should be written on one line, as follows:

```
<!-- This is a comment -->
```

- Comments that span more than one line should be written as follows::

```
<!--
This is an example of a long comment.
This is an example of a long comment.
-->
```

- Long comments are easier to see if they are indented with two spaces.

## Best Practices 5/6

- Use simple syntax to load **external scripts** (the **type** attribute is **not necessary**):

```
<script src="script.js">
```

- Use simple syntax to link **to style sheets** (**type** attribute is **not necessary**):

```
<link rel="stylesheet" href="style.css">
```

## Best Practices 6/6

- Some web servers (Apache, Unix) are case-sensitive in file names: "london.jpg" cannot be accessed as "London.jpg". Other web servers (Microsoft, IIS) are not case-sensitive: "london.jpg" can be obtained as "London.jpg".
  - If you use a combination of uppercase and lowercase letters, you must be aware of this. if you switch from a case-sensitive server to a case-nonsensitive server, even small-case errors will ruin your network!
- To avoid these problems, always use file names in **lowercase**!
- HTML files should have a .html extension!
- If the URL does not specify a file name at the end (such as "https://www.pwr.edu.pl/"), the server only adds a default file name, such as "index.html" or "default.html".
  - if the server is configured with only "index.html" as the default file name, the file must be named "index.html" and not "default.html".
- However, servers can be configured with more than one default file name; typically, you can configure as many default file names as you want.

## Links

The correctness of writing the HTML page code can be checked using the HTML validator:

- [https://www.w3schools.com/html/html\\_validate.html](https://www.w3schools.com/html/html_validate.html)

Difference between HTML (pre-HTML5) and HTML5

- [HTML vs HTML5: 2025 In-Depth Comparison & Modern Web Guide | Talent500 blog](#)

# What was not in the lecture, but exists in HTML5

- Element **<template>** - a hidden element that can be used in the future
  - With the help of Javascript code
- Empty **<wbr>** element - (word break) places of breaking the word (for long words, e.g. technical terms)
- And many other elements
  - Many have also been withdrawn in HTML5: **<blink>**, **<center>**, **<frame>** etc.
- Many attributes:
  - General attributes for a large part of the elements (**height**, **width**, **background** color, etc.)
  - Element-specific
- You can create your **own tag** in JavaScript
  - New element definition code:  
`customElements.define('popup-info', PopUpInfo);`
  - From this point **<popup-info ...=>** is a new tag
  - Second parameter `PopUpInfo` is a new class with support functions for this tag
- Form elements theoretically can be outside the form. Then, however, you definitely need to have code in JavaScript with a response to events appearing in a given element.